

11 Linear Regression

11.1 Introduction

In this chapter, we discuss **linear regression**, which is a very widely used method for predicting a real-valued output (also called the **dependent variable** or **target**) $y \in \mathbb{R}$, given a vector of real-valued inputs (also called **independent variables**, **explanatory variables**, or **covariates**) $\mathbf{x} \in \mathbb{R}^D$. The key property of the model is that the expected value of the output is assumed to be a linear function of the input, $\mathbb{E}[y|\mathbf{x}] = \mathbf{w}^\top \mathbf{x}$, which makes the model easy to interpret, and easy to fit to data. We discuss nonlinear extensions later in this book.

11.2 Least squares linear regression

In this section, we discuss the most common form of linear regression model.

11.2.1 Terminology

The term “linear regression” usually refers to a model of the following form:

$$p(y|\mathbf{x}, \boldsymbol{\theta}) = \mathcal{N}(y|w_0 + \mathbf{w}^\top \mathbf{x}, \sigma^2) \quad (11.1)$$

where $\boldsymbol{\theta} = (w_0, \mathbf{w}, \sigma^2)$ are all the parameters of the model. (In statistics, the parameters w_0 and $\mathbf{w}$ are usually denoted by β_0 and $\boldsymbol{\beta}$.)

The vector of parameters $\mathbf{w}_{1:D}$ are known as the **weights** or **regression coefficients**. Each coefficient w_d specifies the change in the output we expect if we change the corresponding input feature x_d by one unit. For example, suppose x_1 is the age of a person, x_2 is their education level (represented as a continuous number), and y is their income. Thus w_1 corresponds to the increase in income we expect as someone becomes one year older (and hence get more experience), and w_2 corresponds to the increase in income we expect as someone’s education level increases by one level. The term w_0 is the **offset** or **bias** term, and specifies the output value if all the inputs are 0. This captures the unconditional mean of the response, $w_0 = \mathbb{E}[y]$, and acts as a baseline. We will usually assume that $\mathbf{x}$ is written as $[1, x_1, \dots, x_D]$, so we can absorb the offset term w_0 into the weight vector $\mathbf{w}$.

If the input is one-dimensional (so $D = 1$), the model has the form $f(\mathbf{x}; \mathbf{w}) = ax + b$, where $b = w_0$ is the intercept, and $a = w_1$ is the slope. This is called **simple linear regression**. If the input is multi-dimensional, $\mathbf{x} \in \mathbb{R}^D$ where $D > 1$, the method is called **multiple linear regression**. If the

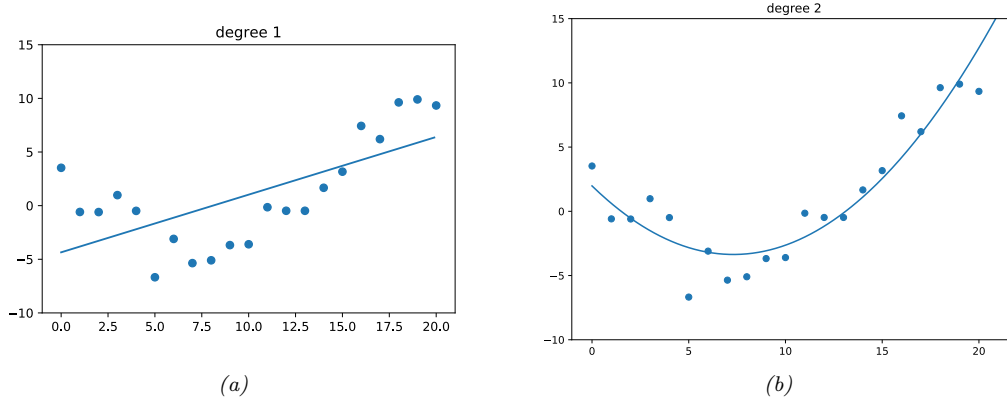


Figure 11.1: Polynomial of degrees 1 and 2 fit to 21 datapoints. Generated by [linreg_poly_vs_degree.ipynb](#).

output is also multi-dimensional, $\mathbf{y} \in \mathbb{R}^J$, where $J > 1$, it is called **multivariate linear regression**,

$$p(\mathbf{y}|\mathbf{x}, \mathbf{W}) = \prod_{j=1}^J \mathcal{N}(y_j | \mathbf{w}_j^\top \mathbf{x}, \sigma_j^2) \quad (11.2)$$

See Exercise 11.1 for a simple numerical example.

In general, a straight line will not provide a good fit to most data sets. However, we can always apply a nonlinear transformation to the input features, by replacing $\mathbf{x}$ with $\phi(\mathbf{x})$ to get

$$p(y|\mathbf{x}, \boldsymbol{\theta}) = \mathcal{N}(y | \mathbf{w}^\top \phi(\mathbf{x}), \sigma^2) \quad (11.3)$$

As long as the parameters of the **feature extractor** ϕ are fixed, the model remains *linear in the parameters*, even if it is not linear in the inputs. (We discuss ways to learn the feature extractor, and the final linear mapping, in Part III.)

As a simple example of a nonlinear transformation, consider the case of **polynomial regression**, which we introduced in Section 1.2.2.2. If the input is 1d, and we use a polynomial expansion of degree d , we get $\phi(x) = [1, x, x^2, \dots, x^d]$. See Figure 11.1 for an example. (See also Section 11.5 where we discuss splines.)

11.2.2 Least squares estimation

To fit a linear regression model to data, we will minimize the negative log likelihood on the training set. The objective function is given by

$$\text{NLL}(\mathbf{w}, \sigma^2) = - \sum_{n=1}^N \log \left[\left(\frac{1}{2\pi\sigma^2} \right)^{\frac{1}{2}} \exp \left(-\frac{1}{2\sigma^2} (y_n - \mathbf{w}^\top \mathbf{x}_n)^2 \right) \right] \quad (11.4)$$

$$= \frac{1}{2\sigma^2} \sum_{n=1}^N (y_n - \hat{y}_n)^2 + \frac{N}{2} \log(2\pi\sigma^2) \quad (11.5)$$

where we have defined the predicted response $\hat{y}_n \triangleq \mathbf{w}^\top \mathbf{x}_n$. The MLE is the point where $\nabla_{\mathbf{w}, \sigma} \text{NLL}(\mathbf{w}, \sigma^2) = \mathbf{0}$. We can first optimize wrt $\mathbf{w}$, and then solve for the optimal σ .

In this section, we just focus on estimating the weights $\mathbf{w}$. In this case, the NLL is equal (up to irrelevant constants) to the **residual sum of squares**, which is given by

$$\text{RSS}(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N (y_n - \mathbf{w}^\top \mathbf{x}_n)^2 = \frac{1}{2} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2 = \frac{1}{2} (\mathbf{X}\mathbf{w} - \mathbf{y})^\top (\mathbf{X}\mathbf{w} - \mathbf{y}) \quad (11.6)$$

We discuss how to optimize this below.

11.2.2.1 Ordinary least squares

From Equation (7.264) we can show that the gradient is given by

$$\nabla_{\mathbf{w}} \text{RSS}(\mathbf{w}) = \mathbf{X}^\top \mathbf{X}\mathbf{w} - \mathbf{X}^\top \mathbf{y} \quad (11.7)$$

Setting the gradient to zero and solving gives

$$\mathbf{X}^\top \mathbf{X}\mathbf{w} = \mathbf{X}^\top \mathbf{y} \quad (11.8)$$

These are known as the **normal equations**, since, at the optimal solution, $\mathbf{y} - \mathbf{X}\mathbf{w}$ is normal (orthogonal) to the range of $\mathbf{X}$, as we explain in Section 11.2.2.2. The corresponding solution $\hat{\mathbf{w}}$ is the **ordinary least squares (OLS)** solution, which is given by

$$\hat{\mathbf{w}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y} \quad (11.9)$$

The quantity $\mathbf{X}^\dagger = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top$ is the (left) pseudo inverse of the (non-square) matrix $\mathbf{X}$ (see Section 7.5.3 for more details).

We can check that the solution is unique by showing that the Hessian is positive definite. In this case, the Hessian is given by

$$\mathbf{H}(\mathbf{w}) = \frac{\partial^2}{\partial \mathbf{w}^2} \text{RSS}(\mathbf{w}) = \mathbf{X}^\top \mathbf{X} \quad (11.10)$$

If $\mathbf{X}$ is full rank (so the columns of $\mathbf{X}$ are linearly independent), then $\mathbf{H}$ is positive definite, since for any $\mathbf{v} > \mathbf{0}$, we have

$$\mathbf{v}^\top (\mathbf{X}^\top \mathbf{X}) \mathbf{v} = (\mathbf{X}\mathbf{v})^\top (\mathbf{X}\mathbf{v}) = \|\mathbf{X}\mathbf{v}\|^2 > 0 \quad (11.11)$$

Hence in the full rank case, the least squares objective has a unique global minimum. See Figure 11.2 for an illustration.

11.2.2.2 Geometric interpretation of least squares

The normal equations have an elegant geometrical interpretation, deriving from Section 7.7, as we now explain. We will assume $N > D$, so there are more observations than unknowns. (This is known

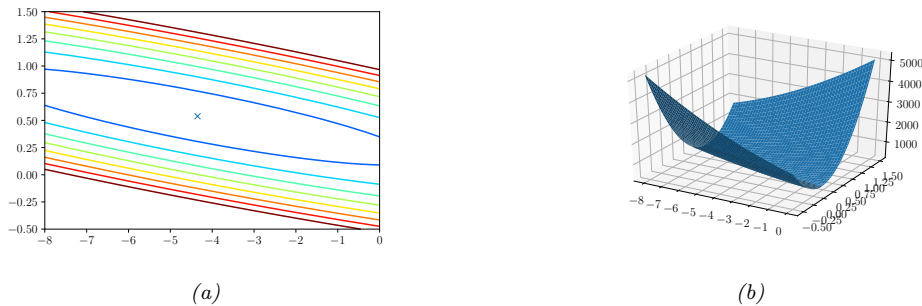


Figure 11.2: (a) Contours of the RSS error surface for the example in Figure 11.1a. The blue cross represents the MLE. (b) Corresponding surface plot. Generated by `linreg_contours_sse_plot.ipynb`.

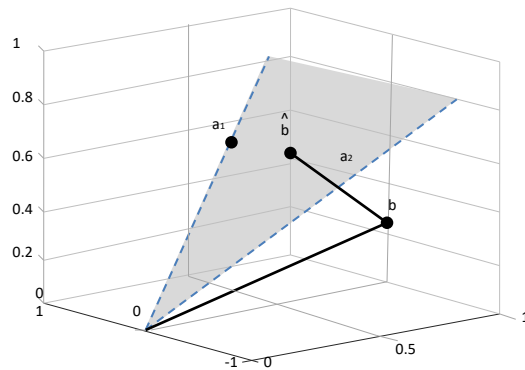


Figure 11.3: Graphical interpretation of least squares for $m = 3$ equations and $n = 2$ unknowns when solving the system $\mathbf{A}\mathbf{x} = \mathbf{b}$. $\mathbf{a}_1$ and $\mathbf{a}_2$ are the columns of $\mathbf{A}$, which define a 2d linear subspace embedded in $\mathbb{R}^3$. The target vector $\mathbf{b}$ is a vector in $\mathbb{R}^3$; its orthogonal projection onto the linear subspace is denoted $\hat{\mathbf{b}}$. The line from $\mathbf{b}$ to $\hat{\mathbf{b}}$ is the vector of residual errors, whose norm we want to minimize.

as an **overdetermined system**.) We seek a vector $\hat{\mathbf{y}} \in \mathbb{R}^N$ that lies in the linear subspace spanned by $\mathbf{X}$ and is as close as possible to $\mathbf{y}$, i.e., we want to find

$$\underset{\hat{\mathbf{y}} \in \text{span}(\{\mathbf{x}_{:,1}, \dots, \mathbf{x}_{:,D}\})}{\text{argmin}} \quad \|\mathbf{y} - \hat{\mathbf{y}}\|_2. \quad (11.12)$$

where $\mathbf{x}_{:,d}$ is the d 'th column of $\mathbf{X}$. Since $\hat{\mathbf{y}} \in \text{span}(\mathbf{X})$, there exists some weight vector $\mathbf{w}$ such that

$$\hat{\mathbf{y}} = w_1 \mathbf{x}_{:,1} + \dots + w_D \mathbf{x}_{:,D} = \mathbf{X}\mathbf{w} \quad (11.13)$$

To minimize the norm of the residual, $\mathbf{y} - \hat{\mathbf{y}}$, we want the residual vector to be orthogonal to every column of $\mathbf{X}$. Hence

$$\mathbf{x}_{:,d}^\top (\mathbf{y} - \hat{\mathbf{y}}) = 0 \Rightarrow \mathbf{X}^\top (\mathbf{y} - \mathbf{X}\mathbf{w}) = \mathbf{0} \Rightarrow \mathbf{w} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y} \quad (11.14)$$

Hence our projected value of $\mathbf{y}$ is given by

$$\hat{\mathbf{y}} = \mathbf{X}\mathbf{w} = \mathbf{X}(\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y} \quad (11.15)$$

This corresponds to an **orthogonal projection** of $\mathbf{y}$ onto the column space of $\mathbf{X}$. For example, consider the case where we have $N = 3$ training examples, each of dimensionality $D = 2$. The training data defines a 2d linear subspace, defined by the 2 columns of $\mathbf{X}$, each of which is a point in 3d. We project $\mathbf{y}$, which is also a point in 3d, onto this 2d subspace, as shown in Figure 11.3.

The **projection matrix**

$$\text{Proj}(\mathbf{X}) \triangleq \mathbf{X}(\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \quad (11.16)$$

is sometimes called the **hat matrix**, since $\hat{\mathbf{y}} = \text{Proj}(\mathbf{X})\mathbf{y}$. In the special case that $\mathbf{X} = \mathbf{x}$ is a column vector, the orthogonal projection of $\mathbf{y}$ onto the line $\mathbf{x}$ becomes

$$\text{Proj}(\mathbf{x})\mathbf{y} = \mathbf{x} \frac{\mathbf{x}^\top \mathbf{y}}{\mathbf{x}^\top \mathbf{x}} \quad (11.17)$$

11.2.2.3 Algorithmic issues

Recall that the OLS solution is

$$\hat{\mathbf{w}} = \mathbf{X}^\dagger \mathbf{y} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y} \quad (11.18)$$

However, even if it is theoretically possible to compute the pseudo-inverse by inverting $\mathbf{X}^\top \mathbf{X}$, we should not do so for numerical reasons, since $\mathbf{X}^\top \mathbf{X}$ may be ill conditioned or singular.

A better (and more general) approach is to compute the pseudo-inverse using the SVD. Indeed, if you look at the source code for the function `sklearn.linear_model.fit`, you will see that it uses the `scipy.linalg.lstsq` function, which in turns calls `DGELS`, which is an SVD-based solver implemented by the `LAPACK` library, written in Fortran.¹

However, if $\mathbf{X}$ is tall and skinny (i.e., $N \gg D$), it can be quicker to use QR decomposition (Section 7.6.2). To do this, let $\mathbf{X} = \mathbf{QR}$, where $\mathbf{Q}^\top \mathbf{Q} = \mathbf{I}$. In Section 7.7, we show that OLS is equivalent to solving the system of linear equations $\mathbf{X}\mathbf{w} = \mathbf{y}$ in a way that minimizes $\|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2$. (If $N = D$ and $\mathbf{X}$ is full rank, the equations have a unique solution, and the error will be 0.) Using QR decomposition, we can rewrite this system of equations as follows:

$$(\mathbf{QR})\mathbf{w} = \mathbf{y} \quad (11.19)$$

$$\mathbf{Q}^\top \mathbf{QR}\mathbf{w} = \mathbf{Q}^\top \mathbf{y} \quad (11.20)$$

$$\mathbf{w} = \mathbf{R}^{-1}(\mathbf{Q}^\top \mathbf{y}) \quad (11.21)$$

Since $\mathbf{R}$ is upper triangular, we can solve this last set of equations using backsubstitution, thus avoiding matrix inversion. See `linsys_solve_demo.ipynb` for a demo.

An alternative to the use of direct methods based on matrix decomposition (such as SVD and QR) is to use iterative solvers, such as the **conjugate gradient** method (which assumes $\mathbf{X}$ is symmetric

1. Note that a lot of the “Python” scientific computing stack sits on top of source code that is written in Fortran or C++, for reasons of speed. This makes it hard to change the underlying algorithms. By contrast, the scientific computing libraries in the `Julia` language are written in Julia itself, aiding clarity without sacrificing speed.

positive definite), and the **GMRES** (generalized minimal residual method), that works for general $\mathbf{X}$. (In SciPy, this is implemented by `sparse.linalg.gmres`.) These methods just require the ability to perform matrix-vector multiplications (i.e., an implementation of a **linear operator**), and thus are well-suited to problems where $\mathbf{X}$ is sparse or structured. For details, see e.g., [TB97].

A final important issue is that it is usually essential to **standardize** the input features before fitting the model, to ensure that they are zero mean and unit variance. We can do this using Equation (10.51).

11.2.2.4 Weighted least squares

In some cases, we want to associate a weight with each example. For example, in **heteroskedastic regression**, the variance depends on the input, so the model has the form

$$p(y|\mathbf{x};\boldsymbol{\theta}) = \mathcal{N}(y|\mathbf{w}^\top \mathbf{x}, \sigma^2(\mathbf{x})) = \frac{1}{\sqrt{2\pi\sigma^2(\mathbf{x})}} \exp\left(-\frac{1}{2\sigma^2(\mathbf{x})} (y - \mathbf{w}^\top \mathbf{x})^2\right) \quad (11.22)$$

Thus

$$p(\mathbf{y}|\mathbf{x};\boldsymbol{\theta}) = \mathcal{N}(\mathbf{y}|\mathbf{X}\mathbf{w}, \boldsymbol{\Lambda}^{-1}) \quad (11.23)$$

where $\boldsymbol{\Lambda} = \text{diag}(1/\sigma^2(\mathbf{x}_n))$. This is known as **weighted linear regression**. One can show that the MLE is given by

$$\hat{\mathbf{w}} = (\mathbf{X}^\top \boldsymbol{\Lambda} \mathbf{X})^{-1} \mathbf{X}^\top \boldsymbol{\Lambda} \mathbf{y} \quad (11.24)$$

This is known as the **weighted least squares** estimate.

11.2.3 Other approaches to computing the MLE

In this section, we discuss other approaches for computing the MLE.

11.2.3.1 Solving for offset and slope separately

Typically we use a model of the form $p(y|\mathbf{x}, \boldsymbol{\theta}) = \mathcal{N}(y|w_0 + \mathbf{w}^\top \mathbf{x}, \sigma^2)$, where w_0 is an offset or “bias” term. We can compute $(w_0, \mathbf{w})$ at the same time by adding a column of 1s to $\mathbf{X}$, and then computing the MLE as above. Alternatively, we can solve for $\mathbf{w}$ and w_0 separately. (This will be useful later.) In particular, one can show that

$$\hat{\mathbf{w}} = (\mathbf{X}_c^\top \mathbf{X}_c)^{-1} \mathbf{X}_c^\top \mathbf{y}_c = \left[\sum_{i=1}^N (\mathbf{x}_n - \bar{\mathbf{x}})(\mathbf{x}_n - \bar{\mathbf{x}})^\top \right]^{-1} \left[\sum_{i=1}^N (y_n - \bar{y})(\mathbf{x}_n - \bar{\mathbf{x}}) \right] \quad (11.25)$$

$$\hat{w}_0 = \frac{1}{N} \sum_n y_n - \frac{1}{N} \sum_n \mathbf{x}_n^\top \hat{\mathbf{w}} = \bar{y} - \bar{\mathbf{x}}^\top \hat{\mathbf{w}} \quad (11.26)$$

where $\mathbf{X}_c$ is the centered input matrix containing $\mathbf{x}_n^c = \mathbf{x}_n - \bar{\mathbf{x}}$ along its rows, and $\mathbf{y}_c = \mathbf{y} - \bar{y}$ is the centered output vector. Thus we can first compute $\hat{\mathbf{w}}$ on centered data, and then estimate w_0 using $\bar{y} - \bar{\mathbf{x}}^\top \hat{\mathbf{w}}$.

Note that, if we write the model in the form $\hat{y} = \beta_0 + \boldsymbol{\beta}^\top (\mathbf{x} - \bar{\mathbf{x}})$, then we have $\hat{\boldsymbol{\beta}} = \hat{\mathbf{w}}$ and $\hat{w}_0 = \hat{\beta}_0 - \hat{\boldsymbol{\beta}}^\top \bar{\mathbf{x}}$, so $\hat{\beta}_0 = \bar{y}$.

11.2.3.2 Simple linear regression (1d inputs)

In the case of 1d (scalar) inputs, the results from Section 11.2.3.1 reduce to the following simple form, which may be familiar from basic statistics classes:

$$\hat{w}_1 = \frac{\sum_n (x_n - \bar{x})(y_n - \bar{y})}{\sum_n (x_n - \bar{x})^2} = \frac{C_{xy}}{C_{xx}} = \rho_{xy} \frac{\sigma_y}{\sigma_x} \quad (11.27)$$

$$\hat{w}_0 = \mathbb{E}[y] - w_1 \mathbb{E}[x] \approx \bar{y} - \hat{w}_1 \bar{x} \quad (11.28)$$

where $C_{xx} = \text{Cov}[X, X] = \mathbb{V}[X] = \sigma_x^2$, $C_{yy} = \text{Cov}[Y, Y] = \mathbb{V}[Y] = \sigma_y^2$, $C_{xy} = \text{Cov}[X, Y]$, and $\rho_{xy} = \frac{C_{xy}}{\sigma_x \sigma_y}$. Hence the prediction becomes $\hat{y} = \bar{y} - \hat{w}_1 \bar{x} + \hat{w}_1 x = \bar{y} + \rho_{xy} \frac{(x - \bar{x})}{\sigma_x}$. We can interpret this equation as follows: we estimate how many standard deviations x is away from the mean of X , and then we predict the outcome to be the mean of Y plus ρ times that many standard deviations above or below.

11.2.3.3 Partial regression

From Equation (11.27), we can compute the **regression coefficient** of Y on X as follows:

$$R_{YX} \triangleq \frac{\partial}{\partial x} \mathbb{E}[Y|X = x] = w_1 = \frac{C_{xy}}{C_{xx}} \quad (11.29)$$

This is the slope of the linear prediction for Y given X .

Now consider the case where we have 2 inputs, so $Y = w_0 + w_1 X_1 + w_2 X_2 + \epsilon$, where $\mathbb{E}[\epsilon] = 0$. One can show that the optimal regression coefficient for w_1 is given by $R_{YX_1 \cdot X_2}$, which is the **partial regression coefficient** of Y on X_1 , keeping X_2 constant:

$$w_1 = R_{YX_1 \cdot X_2} = \frac{\partial}{\partial x} \mathbb{E}[Y|X_1 = x, X_2] \quad (11.30)$$

Note that this quantity is invariant to the specific value of X_2 we condition on.

We can derive w_2 in a similar manner. Indeed, we can extend this to multiple input variables. In each case, we find the optimal coefficients are equal to the partial regression coefficients. This means that we can interpret the j 'th coefficient $\hat{w}_j$ as the change in output y we expect per unit change in input x_j , keeping all the other inputs constant.

11.2.3.4 Recursively computing the MLE

OLS is a batch method for computing the MLE. In some applications, the data arrives in a continual stream, so we want to compute the estimate online, or **recursively**, as we discussed in Section 4.4.2. In this section, we show how to do this for the case of simple (1d) linear regression.

Recall from Section 11.2.3.2 that the batch MLE for simple linear regression is given by

$$\hat{w}_1 = \frac{\sum_n (x_n - \bar{x})(y_n - \bar{y})}{\sum_n (x_n - \bar{x})^2} = \frac{C_{xy}}{C_{xx}} \quad (11.31)$$

$$\hat{w}_0 = \bar{y} - \hat{w}_1 \bar{x} \quad (11.32)$$

where $C_{xy} = \text{Cov}[X, Y]$ and $C_{xx} = \text{Cov}[X, X] = \mathbb{V}[X]$.

We now discuss how to compute these results in a recursive fashion. To do this, let us define the following sufficient statistics:

$$\bar{x}^{(n)} = \frac{1}{n} \sum_{i=1}^n x_i, \quad \bar{y}^{(n)} = \frac{1}{n} \sum_{i=1}^n y_i \quad (11.33)$$

$$C_{xx}^{(n)} = \frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2, \quad C_{xy}^{(n)} = \frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y}), \quad C_{yy}^{(n)} = \frac{1}{n} \sum_{i=1}^n (y_i - \bar{y})^2 \quad (11.34)$$

We can update the means online using

$$\bar{x}^{(n+1)} = \bar{x}^{(n)} + \frac{1}{n+1}(x_{n+1} - \bar{x}^{(n)}), \quad \bar{y}^{(n+1)} = \bar{y}^{(n)} + \frac{1}{n+1}(y_{n+1} - \bar{y}^{(n)}) \quad (11.35)$$

To update the covariance terms, let us first rewrite $C_{xy}^{(n)}$ as follows:

$$C_{xy}^{(n)} = \frac{1}{n} \left[\left(\sum_{i=1}^n x_i y_i \right) + \left(\sum_{i=1}^n \bar{x}^{(n)} \bar{y}^{(n)} \right) - \bar{x}^{(n)} \left(\sum_{i=1}^n y_i \right) - \bar{y}^{(n)} \left(\sum_{i=1}^n x_i \right) \right] \quad (11.36)$$

$$= \frac{1}{n} \left[\left(\sum_{i=1}^n x_i y_i \right) + n \bar{x}^{(n)} \bar{y}^{(n)} - \bar{x}^{(n)} n \bar{y}^{(n)} - \bar{y}^{(n)} n \bar{x}^{(n)} \right] \quad (11.37)$$

$$= \frac{1}{n} \left[\left(\sum_{i=1}^n x_i y_i \right) - n \bar{x}^{(n)} \bar{y}^{(n)} \right] \quad (11.38)$$

Hence

$$\sum_{i=1}^n x_i y_i = n C_{xy}^{(n)} + n \bar{x}^{(n)} \bar{y}^{(n)} \quad (11.39)$$

and so

$$C_{xy}^{(n+1)} = \frac{1}{n+1} \left[x_{n+1} y_{n+1} + n C_{xy}^{(n)} + n \bar{x}^{(n)} \bar{y}^{(n)} - (n+1) \bar{x}^{(n+1)} \bar{y}^{(n+1)} \right] \quad (11.40)$$

We can derive the update for $C_{xx}^{(n+1)}$ in a similar manner.

See Figure 11.4 for a simple illustration of these equations in action for a 1d regression model.

To extend the above analysis to D -dimensional inputs, the easiest approach is to use SGD. The resulting algorithm is called the **least mean squares** algorithm; see Section 8.4.2 for details.

11.2.3.5 Deriving the MLE from a generative perspective

Linear regression is a discriminative model of the form $p(y|\mathbf{x})$. However, we can also use generative models for regression, by analogy to how we use generative models for classification in Chapter 9. The goal is to compute the conditional expectation

$$f(\mathbf{x}) = \mathbb{E}[y|\mathbf{x}] = \int y p(y|\mathbf{x}) dy = \frac{\int y p(\mathbf{x}, y) dy}{\int p(\mathbf{x}, y) dy} \quad (11.41)$$

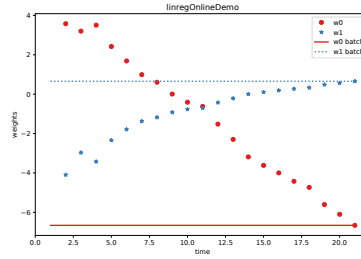


Figure 11.4: Regression coefficients over time for the 1d model in Figure 1.7a(a). Generated by `linregOnlineDemo.ipynb`.

Suppose we fit $p(\mathbf{x}, y)$ using an MVN. The MLEs for the parameters of the joint distribution are the empirical means and covariances (see Section 4.2.6 for a proof of this result):

$$\boldsymbol{\mu}_x = \frac{1}{N} \sum_n \mathbf{x}_n \quad (11.42)$$

$$\mu_y = \frac{1}{N} \sum_n y_n \quad (11.43)$$

$$\boldsymbol{\Sigma}_{xx} = \frac{1}{N} \sum_n (\mathbf{x}_n - \bar{\mathbf{x}})(\mathbf{x}_n - \bar{\mathbf{x}})^\top = \frac{1}{N} \mathbf{X}_c^\top \mathbf{X}_c \quad (11.44)$$

$$\boldsymbol{\Sigma}_{xy} = \frac{1}{N} \sum_n (\mathbf{x}_n - \bar{\mathbf{x}})(y_n - \bar{y}) = \frac{1}{N} \mathbf{X}_c^\top \mathbf{y}_c \quad (11.45)$$

Hence from Equation (3.28), we have

$$\mathbb{E}[y|\mathbf{x}] = \mu_y + \boldsymbol{\Sigma}_{xy}^\top \boldsymbol{\Sigma}_{xx}^{-1}(\mathbf{x} - \boldsymbol{\mu}_x) \quad (11.46)$$

We can rewrite this as $\mathbb{E}[y|\mathbf{x}] = w_0 + \mathbf{w}^\top \mathbf{x}$ by defining

$$w_0 = \mu_y - \mathbf{w}^\top \boldsymbol{\mu}_x = \bar{y} - \mathbf{w}^\top \bar{\mathbf{x}} \quad (11.47)$$

$$\mathbf{w} = \boldsymbol{\Sigma}_{xx}^{-1} \boldsymbol{\Sigma}_{xy} = (\mathbf{X}_c^\top \mathbf{X}_c)^{-1} \mathbf{X}_c^\top \mathbf{y}_c \quad (11.48)$$

This matches the MLEs for the discriminative model as we showed in Section 11.2.3.1. Thus we see that fitting the joint model, and then conditioning it, yields the same result as fitting the conditional model. However, this is only true for Gaussian models (see Section 9.4 for further discussion of this point).

11.2.3.6 Deriving the MLE for σ^2

After estimating $\hat{\mathbf{w}}_{\text{mle}}$ using one of the above methods, we can estimate the noise variance. It is easy to show that the MLE is given by

$$\hat{\sigma}_{\text{mle}}^2 = \underset{\sigma^2}{\text{argmin}} \text{NLL}(\hat{\mathbf{w}}, \sigma^2) = \frac{1}{N} \sum_{n=1}^N (y_n - \mathbf{x}_n^\top \hat{\mathbf{w}})^2 \quad (11.49)$$

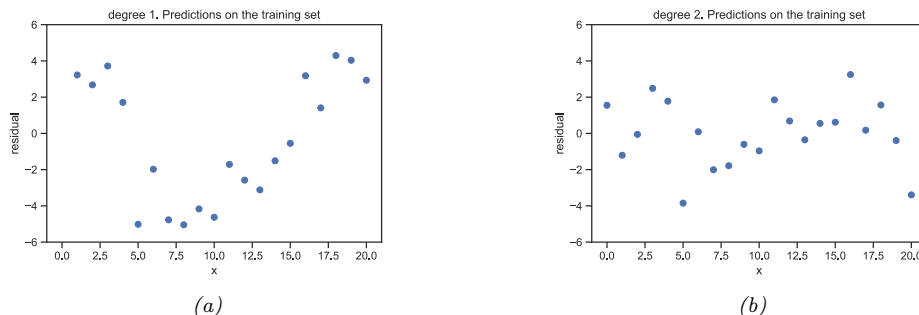


Figure 11.5: Residual plot for polynomial regression of degree 1 and 2 for the functions in Figure 1.7a(a-b). Generated by `linreg_poly_vs_degree.ipynb`.

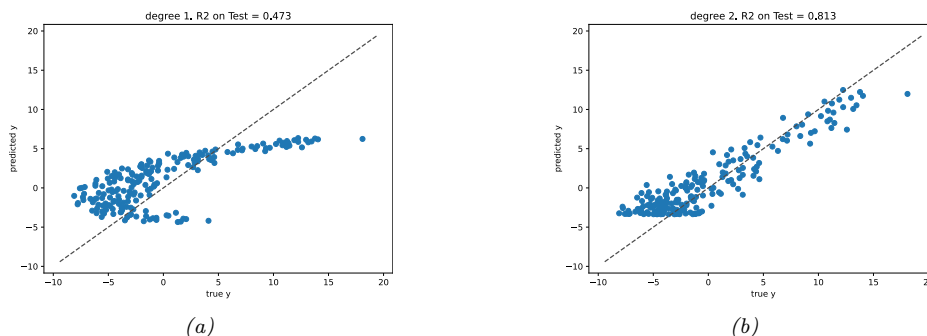


Figure 11.6: Fit vs actual plots for polynomial regression of degree 1 and 2 for the functions in Figure 1.7a(a-b). Generated by `linreg_poly_vs_degree.ipynb`.

This is just the MSE of the residuals, which is an intuitive result.

11.2.4 Measuring goodness of fit

In this section, we discuss some simple ways to assess how well a regression model fits the data (which is known as **goodness of fit**).

11.2.4.1 Residual plots

For 1d inputs, we can check the reasonableness of the model by plotting the residuals, $r_n = y_n - \hat{y}_n$, vs the input x_n . This is called a **residual plot**. The model assumes that the residuals have a $\mathcal{N}(0, \sigma^2)$ distribution, so the residual plot should be a cloud of points more or less equally above and below the horizontal line at 0, without any obvious trends.

As an example, in Figure 11.5(a), we plot the residuals for the linear model in Figure 1.7a(a). We see that there is some curved structure to the residuals, indicating a lack of fit. In Figure 11.5(b), we plot the residuals for the quadratic model in Figure 1.7a(b). We see a much better fit.

To extend this approach to multi-dimensional inputs, we can plot predictions $\hat{y}_n$ vs the true output y_n , rather than plotting vs x_n . A good model will have points that lie on a diagonal line. See Figure 11.6 for some examples.

11.2.4.2 Prediction accuracy and R^2

We can assess the fit quantitatively by computing the RSS (residual sum of squares) on the dataset: $\text{RSS}(\mathbf{w}) = \sum_{n=1}^N (y_n - \mathbf{w}^\top \mathbf{x}_n)^2$. A model with lower RSS fits the data better. Another measure that is used is **root mean squared error** or **RMSE**:

$$\text{RMSE}(\mathbf{w}) \triangleq \sqrt{\frac{1}{N} \text{RSS}(\mathbf{w})} \quad (11.50)$$

A more interpretable measure can be computed using the **coefficient of determination**, denoted by R^2 :

$$R^2 \triangleq 1 - \frac{\sum_{n=1}^N (\hat{y}_n - y_n)^2}{\sum_{n=1}^N (\bar{y} - y_n)^2} = 1 - \frac{\text{RSS}}{\text{TSS}} \quad (11.51)$$

where $\bar{y} = \frac{1}{N} \sum_{n=1}^N y_n$ is the empirical mean of the response, $\text{RSS} = \sum_{n=1}^N (y_n - \hat{y}_n)^2$ is the residual sum of squares, and $\text{TSS} = \sum_{n=1}^N (y_n - \bar{y})^2$ is the total sum of squares. Thus we see that R^2 measures the variance in the predictions relative to a simple constant prediction of $\hat{y}_n = \bar{y}$. If a model does no better at predicting than using the mean of the output, it we have $R^2 = 0$. If the model perfectly fits the data, then the RSS will be 0, so $R^2 = 1$. In general, larger values imply greater reduction in variance (better fit). This is illustrated in Figure 11.6.

11.3 Ridge regression

Maximum likelihood estimation can result in overfitting, as we discussed in Section 1.2.2.2. A simple solution to this is to use MAP estimation with a zero-mean Gaussian prior on the weights, $p(\mathbf{w}) = \mathcal{N}(\mathbf{w}|\mathbf{0}, \lambda^{-1}\mathbf{I})$, as we discussed in Section 4.5.3. This is called **ridge regression**.

In more detail, we compute the MAP estimate as follows:

$$\hat{\mathbf{w}}_{\text{map}} = \underset{\mathbf{w}}{\text{argmin}} \frac{1}{2\sigma^2} (\mathbf{y} - \mathbf{X}\mathbf{w})^\top (\mathbf{y} - \mathbf{X}\mathbf{w}) + \frac{1}{2\tau^2} \mathbf{w}^\top \mathbf{w} \quad (11.52)$$

$$= \underset{\mathbf{w}}{\text{argmin}} \text{RSS}(\mathbf{w}) + \lambda \|\mathbf{w}\|_2^2 \quad (11.53)$$

where $\lambda \triangleq \frac{\sigma^2}{\tau^2}$ is proportional to the strength of the prior, and

$$\|\mathbf{w}\|_2 \triangleq \sqrt{\sum_{d=1}^D |w_d|^2} = \sqrt{\mathbf{w}^\top \mathbf{w}} \quad (11.54)$$

is the ℓ_2 norm of the vector $\mathbf{w}$. Thus we are penalizing weights that become too large in magnitude. In general, this technique is called ℓ_2 **regularization** or **weight decay**, and is very widely used. See Figure 4.5 for an illustration.

Note that we do not penalize the offset term w_0 , since that only affects the global mean of the output, and does not contribute to overfitting. See Exercise 11.2.

11.3.1 Computing the MAP estimate

In this section, we discuss algorithms for computing the MAP estimate.

The MAP estimate corresponds to minimizing the following penalized objective:

$$J(\mathbf{w}) = (\mathbf{y} - \mathbf{X}\mathbf{w})^\top (\mathbf{y} - \mathbf{X}\mathbf{w}) + \lambda \|\mathbf{w}\|_2^2 \quad (11.55)$$

where $\lambda = \sigma^2/\tau^2$ is the strength of the regularizer. The derivative is given by

$$\nabla_{\mathbf{w}} J(\mathbf{w}) = 2(\mathbf{X}^\top \mathbf{X} \mathbf{w} - \mathbf{X}^\top \mathbf{y} + \lambda \mathbf{w}) \quad (11.56)$$

and hence

$$\hat{\mathbf{w}}_{\text{map}} = (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I}_D)^{-1} \mathbf{X}^\top \mathbf{y} = \left(\sum_n \mathbf{x}_n \mathbf{x}_n^\top + \lambda \mathbf{I}_D \right)^{-1} \left(\sum_n y_n \mathbf{x}_n \right) \quad (11.57)$$

11.3.1.1 Solving using QR

Naively computing the primal estimate $\mathbf{w} = (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^\top \mathbf{y}$ using matrix inversion is a bad idea, since it can be slow and numerically unstable. In this section, we describe a way to convert the problem to a standard least squares problem, to which we can apply QR decomposition, as discussed in Section 11.2.2.3.

We assume the prior has the form $p(\mathbf{w}) = \mathcal{N}(\mathbf{0}, \mathbf{\Lambda}^{-1})$, where $\mathbf{\Lambda}$ is the precision matrix. In the case of ridge regression, $\mathbf{\Lambda} = (1/\tau^2)\mathbf{I}$. We can emulate this prior by adding “virtual data” to the training set to get

$$\tilde{\mathbf{X}} = \begin{pmatrix} \mathbf{X}/\sigma \\ \sqrt{\mathbf{\Lambda}} \end{pmatrix}, \quad \tilde{\mathbf{y}} = \begin{pmatrix} \mathbf{y}/\sigma \\ \mathbf{0}_{D \times 1} \end{pmatrix} \quad (11.58)$$

where $\mathbf{\Lambda} = \sqrt{\mathbf{\Lambda}} \sqrt{\mathbf{\Lambda}}^\top$ is a Cholesky decomposition of $\mathbf{\Lambda}$. We see that $\tilde{\mathbf{X}}$ is $(N + D) \times D$, where the extra rows represent pseudo-data from the prior.

We now show that the RSS on this expanded data is equivalent to penalized RSS on the original data:

$$f(\mathbf{w}) = (\tilde{\mathbf{y}} - \tilde{\mathbf{X}}\mathbf{w})^\top (\tilde{\mathbf{y}} - \tilde{\mathbf{X}}\mathbf{w}) \quad (11.59)$$

$$= \left(\begin{pmatrix} \mathbf{y}/\sigma \\ \mathbf{0} \end{pmatrix} - \begin{pmatrix} \mathbf{X}/\sigma \\ \sqrt{\mathbf{\Lambda}} \end{pmatrix} \mathbf{w} \right)^\top \left(\begin{pmatrix} \mathbf{y}/\sigma \\ \mathbf{0} \end{pmatrix} - \begin{pmatrix} \mathbf{X}/\sigma \\ \sqrt{\mathbf{\Lambda}} \end{pmatrix} \mathbf{w} \right) \quad (11.60)$$

$$= \begin{pmatrix} \frac{1}{\sigma}(\mathbf{y} - \mathbf{X}\mathbf{w}) \\ -\sqrt{\mathbf{\Lambda}}\mathbf{w} \end{pmatrix}^\top \begin{pmatrix} \frac{1}{\sigma}(\mathbf{y} - \mathbf{X}\mathbf{w}) \\ -\sqrt{\mathbf{\Lambda}}\mathbf{w} \end{pmatrix} \quad (11.61)$$

$$= \frac{1}{\sigma^2} (\mathbf{y} - \mathbf{X}\mathbf{w})^\top (\mathbf{y} - \mathbf{X}\mathbf{w}) + (\sqrt{\mathbf{\Lambda}}\mathbf{w})^\top (\sqrt{\mathbf{\Lambda}}\mathbf{w}) \quad (11.62)$$

$$= \frac{1}{\sigma^2} (\mathbf{y} - \mathbf{X}\mathbf{w})^\top (\mathbf{y} - \mathbf{X}\mathbf{w}) + \mathbf{w}^\top \mathbf{\Lambda} \mathbf{w} \quad (11.63)$$

Hence the MAP estimate is given by

$$\hat{\mathbf{w}}_{\text{map}} = (\tilde{\mathbf{X}}^\top \tilde{\mathbf{X}})^{-1} \tilde{\mathbf{X}}^\top \tilde{\mathbf{y}} \quad (11.64)$$

which can be solved using standard OLS methods. In particular, we can compute the QR decomposition of $\tilde{\mathbf{X}}$, and then proceed as in Section 11.2.2.3. This takes $O((N + D)D^2)$ time.

11.3.1.2 Solving using SVD

In this section, we assume $D > N$, which is the usual case when using ridge regression. In this case, it is faster to use SVD than QR. To see how this works, let $\mathbf{X} = \mathbf{U}\mathbf{S}\mathbf{V}^\top$ be the SVD of $\mathbf{X}$, where $\mathbf{V}^\top\mathbf{V} = \mathbf{I}_N$, $\mathbf{U}\mathbf{U}^\top = \mathbf{U}^\top\mathbf{U} = \mathbf{I}_N$, and $\mathbf{S}$ is a diagonal $N \times N$ matrix. Now let $\mathbf{R} = \mathbf{U}\mathbf{S}$ be an $N \times N$ matrix. One can show (see Exercise 18.4 of [HTF09]) that

$$\hat{\mathbf{w}}_{\text{map}} = \mathbf{V}(\mathbf{R}^\top\mathbf{R} + \lambda\mathbf{I}_N)^{-1}\mathbf{R}^\top\mathbf{y} \quad (11.65)$$

In other words, we can replace the D -dimensional vectors $\mathbf{x}_i$ with the N -dimensional vectors $\mathbf{r}_i$ and perform our penalized fit as before. The overall time is now $O(DN^2)$ operations, which is less than $O(D^3)$ if $D > N$.

11.3.2 Connection between ridge regression and PCA

In this section, we discuss an interesting connection between ridge regression and PCA (which we describe in Section 20.1), in order to gain further insight into why ridge regression works well. Our discussion is based on [HTF09, p66].

Let $\mathbf{X} = \mathbf{U}\mathbf{S}\mathbf{V}^\top$ be the SVD of $\mathbf{X}$, where $\mathbf{V}^\top\mathbf{V} = \mathbf{I}_N$, $\mathbf{U}\mathbf{U}^\top = \mathbf{U}^\top\mathbf{U} = \mathbf{I}_N$, and $\mathbf{S}$ is a diagonal $N \times N$ matrix. Using Equation (11.65) we can see that the ridge predictions on the training set are given by

$$\hat{\mathbf{y}} = \mathbf{X}\hat{\mathbf{w}}_{\text{map}} = \mathbf{U}\mathbf{S}\mathbf{V}^\top\mathbf{V}(\mathbf{S}^2 + \lambda\mathbf{I})^{-1}\mathbf{S}\mathbf{U}^\top\mathbf{y} \quad (11.66)$$

$$= \mathbf{U}\tilde{\mathbf{S}}\mathbf{U}^\top\mathbf{y} = \sum_{j=1}^D \mathbf{u}_j \tilde{S}_{jj} \mathbf{u}_j^\top \mathbf{y} \quad (11.67)$$

where

$$\tilde{S}_{jj} \triangleq [\mathbf{S}(\mathbf{S}^2 + \lambda\mathbf{I})^{-1}\mathbf{S}]_{jj} = \frac{\sigma_j^2}{\sigma_j^2 + \lambda} \quad (11.68)$$

and σ_j are the singular values of $\mathbf{X}$. Hence

$$\hat{\mathbf{y}} = \mathbf{X}\hat{\mathbf{w}}_{\text{map}} = \sum_{j=1}^D \mathbf{u}_j \frac{\sigma_j^2}{\sigma_j^2 + \lambda} \mathbf{u}_j^\top \mathbf{y} \quad (11.69)$$

In contrast, the least squares prediction is

$$\hat{\mathbf{y}} = \mathbf{X}\hat{\mathbf{w}}_{\text{mle}} = (\mathbf{U}\mathbf{S}\mathbf{V}^\top)(\mathbf{V}\mathbf{S}^{-1}\mathbf{U}^\top\mathbf{y}) = \mathbf{U}\mathbf{U}^\top\mathbf{y} = \sum_{j=1}^D \mathbf{u}_j \mathbf{u}_j^\top \mathbf{y} \quad (11.70)$$

If σ_j^2 is large compared to λ , then $\sigma_j^2/(\sigma_j^2 + \lambda) \approx \sigma_j^2/\sigma_j^2 = 1$, so direction $\mathbf{u}_j$ is not affected, but if σ_j^2 is small compared to λ , and if λ is large, then $\sigma_j^2/(\sigma_j^2 + \lambda) \approx 1/\lambda \approx 0$, so direction $\mathbf{u}_j$ will be downweighted. In view of this, we define the effective number of **degrees of freedom** of the model as follows:

$$\text{dof}(\lambda) = \sum_{j=1}^D \frac{\sigma_j^2}{\sigma_j^2 + \lambda} \quad (11.71)$$

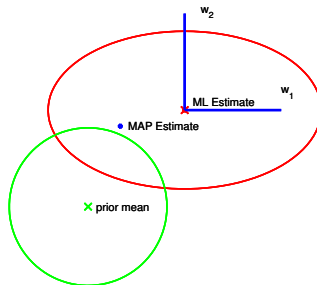


Figure 11.7: Geometry of ridge regression. The likelihood is shown as an ellipse, and the prior is shown as a circle centered on the origin. Adapted from Figure 3.15 of [Bis06]. Generated by `geom_ridge.ipynb`.

When $\lambda = 0$, $\text{dof}(\lambda) = D$, and as $\lambda \rightarrow \infty$, $\text{dof}(\lambda) \rightarrow 0$.

Let us try to understand why this behavior is desirable. In Section 11.7, we show that $\text{Cov}[\mathbf{w}|\mathcal{D}] \propto (\mathbf{X}^T \mathbf{X})^{-1}$, if we use a uniform prior for $\mathbf{w}$. Thus the directions in which we are most uncertain about $\mathbf{w}$ are determined by the eigenvectors of $(\mathbf{X}^T \mathbf{X})^{-1}$ with the largest eigenvalues, as shown in Figure 7.6. These directions correspond to the eigenvectors of $\mathbf{X}^T \mathbf{X}$ with the smallest eigenvalues, and hence (from Section 7.5.2) the smallest singular values. So if σ_j^2 is small relative to λ , ridge regression will downweight direction $\mathbf{u}_j$.

This process is illustrated in Figure 11.7. The horizontal w_1 parameter is not-well determined by the data (has high posterior variance), but the vertical w_2 parameter is well-determined. Hence $w_{\text{map}}(2)$ is close to $w_{\text{mle}}(2)$, but $w_{\text{map}}(1)$ is shifted strongly towards the prior mean, which is 0. In this way, ill-determined parameters are reduced in size towards 0. This is called **shrinkage**.

There is a related, but different, technique called **principal components regression**, which is a supervised version of PCA, which we explain in Section 20.1. The idea is this: first use PCA to reduce the dimensionality to K dimensions, and then use these low dimensional features as input to regression. However, this technique does not work as well as ridge regression in terms of predictive accuracy [HTF01, p70]. The reason is that in PC regression, only the first K (derived) dimensions are retained, and the remaining $D - K$ dimensions are entirely ignored. By contrast, ridge regression uses a “soft” weighting of all the dimensions.

11.3.3 Choosing the strength of the regularizer

To find the optimal value of λ , we can try a finite number of distinct values, and use cross validation to estimate their expected loss, as discussed in Section 4.5.5.2. See Figure 4.5d for an example.

This approach can be quite expensive if we have many values to choose from. Fortunately, we can often **warm start** the optimization procedure, using the value of $\hat{\mathbf{w}}(\lambda_k)$ as an initializer for $\hat{\mathbf{w}}(\lambda_{k+1})$, where $\lambda_{k+1} < \lambda_k$; in other words, we start with a highly constrained model (strong regularizer), and then gradually relax the constraints (decrease the amount of regularization). The set of parameters $\hat{\mathbf{w}}_k$ that we sweep out in this way is known as the **regularization path**. See Figure 11.10(a) for an example.

We can also use an empirical Bayes approach to choose λ . In particular, we choose the hyperparameter by computing $\hat{\lambda} = \arg\max_{\lambda} \log p(\mathcal{D}|\lambda)$, where $p(\mathcal{D}|\lambda)$ is the marginal likelihood or evidence.

Figure 4.7b shows that this gives essentially the same result as the CV estimate. However, the Bayesian approach has several advantages: computing $p(\mathcal{D}|\lambda)$ can be done by fitting a single model, whereas CV has to fit the same model K times; and $p(\mathcal{D}|\lambda)$ is a smooth function of λ , so we can use gradient-based optimization instead of discrete search.

11.4 Lasso regression

In Section 11.3, we assumed a Gaussian prior for the regression coefficients when fitting linear regression models. This is often a good choice, since it encourages the parameters to be small, and hence prevents overfitting. However, sometimes we want the parameters to not just be small, but to be exactly zero, i.e., we want $\hat{\mathbf{w}}$ to be **sparse**, so that we minimize the **L0-norm**:

$$\|\mathbf{w}\|_0 = \sum_{d=1}^D \mathbb{I}(|w_d| > 0) \quad (11.72)$$

This is useful because it can be used to perform **feature selection**. To see this, note that the prediction has the form $f(\mathbf{x}; \mathbf{w}) = \sum_{d=1}^D w_d x_d$, so if any $w_d = 0$, we ignore the corresponding feature x_d . (The same idea can be applied to nonlinear models, such as DNNs, by encouraging the first layer weights to be sparse.)

11.4.1 MAP estimation with a Laplace prior (ℓ_1 regularization)

There are many ways to compute such sparse estimates (see e.g., [Bha+19]). In this section we focus on MAP estimation using the Laplace distribution (which we discussed in Section 11.6.1) as the prior:

$$p(\mathbf{w}|\lambda) = \prod_{d=1}^D \text{Laplace}(w_d|0, 1/\lambda) \propto \prod_{d=1}^D e^{-\lambda|w_d|} \quad (11.73)$$

where λ is the sparsity parameter, and

$$\text{Laplace}(w|\mu, b) \triangleq \frac{1}{2b} \exp\left(-\frac{|w - \mu|}{b}\right) \quad (11.74)$$

Here μ is a location parameter and $b > 0$ is a scale parameter. Figure 2.15 shows that $\text{Laplace}(w|0, b)$ puts more density on 0 than $\mathcal{N}(w|0, \sigma^2)$, even when we fix the variance to be the same.

To perform MAP estimation of a linear regression model with this prior, we just have to minimize the following objective:

$$\text{PNLL}(\mathbf{w}) = -\log p(\mathcal{D}|\mathbf{w}) - \log p(\mathbf{w}|\lambda) = \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2 + \lambda\|\mathbf{w}\|_1 \quad (11.75)$$

where $\|\mathbf{w}\|_1 \triangleq \sum_{d=1}^D |w_d|$ is the ℓ_1 norm of $\mathbf{w}$. This method is called **lasso**, which stands for “least absolute shrinkage and selection operator” [Tib96]. (We explain the reason for this name below.) More generally, MAP estimation with a Laplace prior is called **ℓ_1 -regularization**.

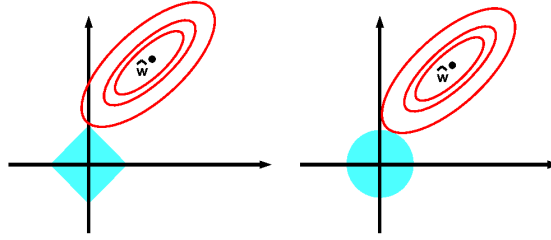


Figure 11.8: Illustration of ℓ_1 (left) vs ℓ_2 (right) regularization of a least squares problem. Adapted from Figure 3.12 of [HTF01].

Note also that we could use other norms for the weight vector. In general, the q -norm is defined as follows:

$$\|\mathbf{w}\|_q = \left(\sum_{d=1}^D |w_d|^q \right)^{1/q} \quad (11.76)$$

For $q < 1$, we can get even sparser solutions. In the limit where $q = 0$, we get the ℓ_0 -**norm**:

$$\|\mathbf{w}\|_0 = \sum_{d=1}^D \mathbb{I}(|w_d| > 0) \quad (11.77)$$

However, one can show that for any $q < 1$, the problem becomes non-convex (see e.g., [HTW15]). Thus ℓ_1 -norm is the tightest **convex relaxation** of the ℓ_0 -norm.

11.4.2 Why does ℓ_1 regularization yield sparse solutions?

We now explain why ℓ_1 regularization results in sparse solutions, whereas ℓ_2 regularization does not. We focus on the case of linear regression, although similar arguments hold for other models.

The lasso objective is the following non-smooth objective (see Section 8.1.4 for a discussion of smoothness):

$$\min_{\mathbf{w}} \text{NLL}(\mathbf{w}) + \lambda \|\mathbf{w}\|_1 \quad (11.78)$$

This is the Lagrangian for the following quadratic program (see Section 8.5.4):

$$\min_{\mathbf{w}} \text{NLL}(\mathbf{w}) \quad \text{s.t.} \quad \|\mathbf{w}\|_1 \leq B \quad (11.79)$$

where B is an upper bound on the ℓ_1 -norm of the weights: a small (tight) bound B corresponds to a large penalty λ , and vice versa.

Similarly, we can write the ridge regression objective $\min_{\mathbf{w}} \text{NLL}(\mathbf{w}) + \lambda \|\mathbf{w}\|_2^2$ in bound constrained form:

$$\min_{\mathbf{w}} \text{NLL}(\mathbf{w}) \quad \text{s.t.} \quad \|\mathbf{w}\|_2^2 \leq B \quad (11.80)$$

In Figure 11.8, we plot the contours of the NLL objective function, as well as the contours of the ℓ_2 and ℓ_1 constraint surfaces. From the theory of constrained optimization (Section 8.5) we know that the optimal solution occurs at the point where the lowest level set of the objective function intersects the constraint surface (assuming the constraint is active). It should be geometrically clear that as we relax the constraint B , we “grow” the ℓ_1 “ball” until it meets the objective; the corners of the ball are more likely to intersect the ellipse than one of the sides, especially in high dimensions, because the corners “stick out” more. The corners correspond to sparse solutions, which lie on the coordinate axes. By contrast, when we grow the ℓ_2 ball, it can intersect the objective at any point; there are no “corners”, so there is no preference for sparsity.

11.4.3 Hard vs soft thresholding

The lasso objective has the form $\mathcal{L}(\mathbf{w}) = \text{NLL}(\mathbf{w}) + \lambda \|\mathbf{w}\|_1$. One can show (Exercise 11.3) that the gradient for the smooth NLL part is given by

$$\frac{\partial}{\partial w_d} \text{NLL}(\mathbf{w}) = a_d w_d - c_d \quad (11.81)$$

$$a_d = \sum_{n=1}^N x_{nd}^2 \quad (11.82)$$

$$c_d = \sum_{n=1}^N x_{nd}(y_n - \mathbf{w}_{-d}^\top \mathbf{x}_{n,-d}) \quad (11.83)$$

where $\mathbf{w}_{-d}$ is $\mathbf{w}$ without component d , and similarly $\mathbf{x}_{n,-d}$ is feature vector $\mathbf{x}_n$ without component d . We see that c_d is proportional to the correlation between d 'th column of features, $\mathbf{x}_{:,d}$, and the residual error obtained by predicting using all the other features, $\mathbf{r}_{-d} = \mathbf{y} - \mathbf{X}_{:,-d} \mathbf{w}_{-d}$. Hence the magnitude of c_d is an indication of how relevant feature d is for predicting $\mathbf{y}$, relative to the other features and the current parameters. Setting the gradient to 0 gives the optimal update for w_d , keeping all other weights fixed:

$$w_d = c_d / a_d = \frac{\mathbf{x}_{:,d}^\top \mathbf{r}_{-d}}{\|\mathbf{x}_{:,d}\|_2^2} \quad (11.84)$$

The corresponding new prediction for $\mathbf{r}_{-d}$ becomes $\hat{\mathbf{r}}_{-d} = w_d \mathbf{x}_{:,d}$, which is the orthogonal projection of the residual onto the column vector $\mathbf{x}_{:,d}$, consistent with Equation (11.15).

Now we add in the ℓ_1 term. Unfortunately, the $\|\mathbf{w}\|_1$ term is not differentiable whenever $w_d = 0$. Fortunately, we can still compute a subgradient at this point. Using Equation (8.14) we find that

$$\partial_{w_d} \mathcal{L}(\mathbf{w}) = (a_d w_d - c_d) + \lambda \partial_{w_d} \|\mathbf{w}\|_1 \quad (11.85)$$

$$= \begin{cases} \{a_d w_d - c_d - \lambda\} & \text{if } w_d < 0 \\ [-c_d - \lambda, -c_d + \lambda] & \text{if } w_d = 0 \\ \{a_d w_d - c_d + \lambda\} & \text{if } w_d > 0 \end{cases} \quad (11.86)$$

Depending on the value of c_d , the solution to $\partial_{w_d} \mathcal{L}(\mathbf{w}) = 0$ can occur at 3 different values of w_d , as follows:

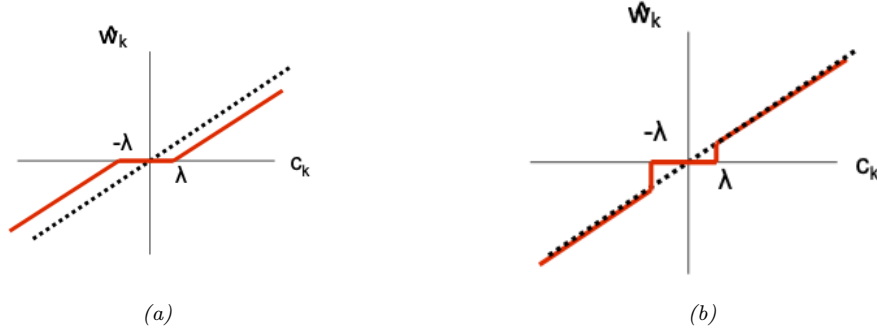


Figure 11.9: Left: soft thresholding. Right: hard thresholding. In both cases, the horizontal axis is the residual error incurred by making predictions using all the coefficients except for w_k , and the vertical axis is the estimated coefficient $\hat{w}_k$ that minimizes this penalized residual. The flat region in the middle is the interval $[-\lambda, +\lambda]$.

1. If $c_d < -\lambda$, so the feature is strongly negatively correlated with the residual, then the subgradient is zero at $\hat{w}_d = \frac{c_d + \lambda}{a_d} < 0$.
2. If $c_d \in [-\lambda, \lambda]$, so the feature is only weakly correlated with the residual, then the subgradient is zero at $\hat{w}_d = 0$.
3. If $c_d > \lambda$, so the feature is strongly positively correlated with the residual, then the subgradient is zero at $\hat{w}_d = \frac{c_d - \lambda}{a_d} > 0$.

In summary, we have

$$\hat{w}_d(c_d) = \begin{cases} (c_d + \lambda)/a_d & \text{if } c_d < -\lambda \\ 0 & \text{if } c_d \in [-\lambda, \lambda] \\ (c_d - \lambda)/a_d & \text{if } c_d > \lambda \end{cases} \quad (11.87)$$

We can write this as follows:

$$\hat{w}_d = \text{SoftThreshold}\left(\frac{c_d}{a_d}, \lambda/a_d\right) \quad (11.88)$$

where

$$\text{SoftThreshold}(x, \delta) \triangleq \text{sign}(x) (|x| - \delta)_+ \quad (11.89)$$

and $x_+ = \max(x, 0)$ is the positive part of x . This is called **soft thresholding** (see also Section 8.6.2). This is illustrated in Figure 11.9(a), where we plot $\hat{w}_d$ vs c_d . The dotted black line is the line $w_d = c_d/a_d$ corresponding to the least squares fit. The solid red line, which represents the regularized estimate $\hat{w}_d$, shifts the dotted line down (or up) by λ , except when $-\lambda \leq c_d \leq \lambda$, in which case it sets $w_d = 0$.

By contrast, in Figure 11.9(b), we illustrate **hard thresholding**. This sets values of w_d to 0 if $-\lambda \leq c_d \leq \lambda$, but it does not shrink the values of w_d outside of this interval. The slope of the soft

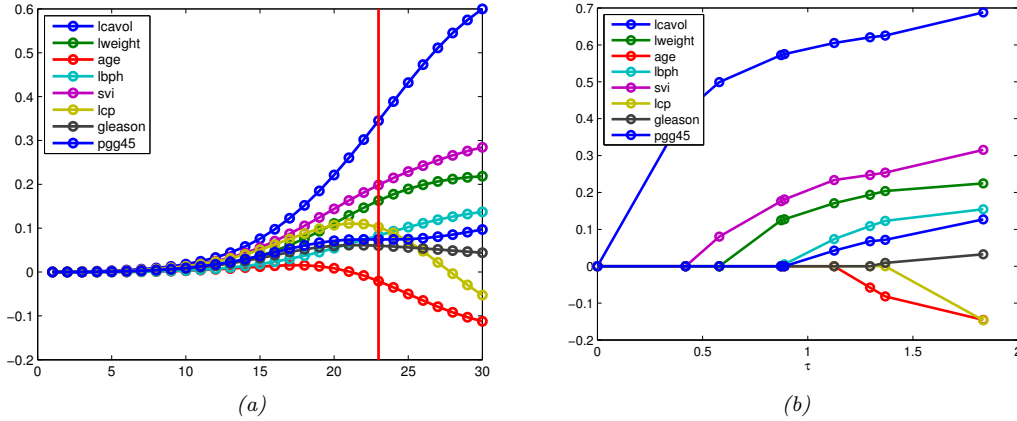


Figure 11.10: (a) Profiles of ridge coefficients for the prostate cancer example vs bound B on ℓ_2 norm of $\mathbf{w}$, so small B (large λ) is on the left. The vertical line is the value chosen by 5-fold CV using the 1 standard error rule. Adapted from Figure 3.8 of [HTF09]. Generated by `ridgePathProstate.ipynb`. (b) Same as (a) but using ℓ_1 norm of $\mathbf{w}$. The x-axis shows the critical values of $\lambda = 1/B$, where the regularization path is discontinuous. Adapted from Figure 3.10 of [HTF09]. Generated by `lassoPathProstate.ipynb`.

thresholding line does not coincide with the diagonal, which means that even large coefficients are shrunk towards zero. This is why lasso stands for “least absolute selection *and shrinkage* operator”. Consequently, lasso is a biased estimator (see Section 4.7.6.1).

A simple solution to the biased estimate problem, known as **debiasing**, is to use a two-stage estimation process: we first estimate the support of the weight vector (i.e., identify which elements are non-zero) using lasso; we then re-estimate the chosen coefficients using least squares. For an example of this in action, see Figure 11.13.

11.4.4 Regularization path

If $\lambda = 0$, we get the OLS solution, which will be dense. As we increase λ , the solution vector $\hat{\mathbf{w}}(\lambda)$ will tend to get sparser. If λ is bigger than some critical value, we get $\hat{\mathbf{w}} = \mathbf{0}$. This critical value is obtained when the gradient of the NLL cancels out with the gradient of the penalty:

$$\lambda_{\max} = \max_d |\nabla_{w_d} \text{NLL}(\mathbf{0})| = \max_d c_d(\mathbf{w} = \mathbf{0}) = \max_d |\mathbf{y}^\top \mathbf{x}_{:,d}| = \|\mathbf{X}^\top \mathbf{y}\|_\infty \quad (11.90)$$

Alternatively, we can work with the bound B on the ℓ_1 norm. When $B = 0$, we get $\hat{\mathbf{w}} = \mathbf{0}$. As we increase B , the solution becomes denser. The largest value of B for which any component is zero is given by $B_{\max} = \|\hat{\mathbf{w}}_{\text{mle}}\|_1$.

As we increase λ , the solution vector $\hat{\mathbf{w}}$ gets sparser, although not necessarily monotonically. We can plot the values $\hat{w}_d$ vs λ (or vs the bound B) for each feature d ; this is known as the **regularization path**. This is illustrated in Figure 11.10(b), where we apply lasso to the prostate cancer regression dataset from [HTF09]. (We treat features `gleason` and `svi` as numeric, not categorical.) On the left,

0	0	0	0	0	0	0	0
0.4279	0	0	0	0	0	0	0
0.5015	0.0735	0	0	0	0	0	0
0.5610	0.1878	0	0	0.0930	0	0	0
0.5622	0.1890	0	0.0036	0.0963	0	0	0
0.5797	0.2456	0	0.1435	0.2003	0	0	0.0901
0.5864	0.2572	-0.0321	0.1639	0.2082	0	0	0.1066
0.6994	0.2910	-0.1337	0.2062	0.3003	-0.2565	0	0.2452
0.7164	0.2926	-0.1425	0.2120	0.3096	-0.2890	-0.0209	0.2773

Table 11.1: Values of the coefficients for linear regression model fit to prostate cancer dataset as we vary the strength of the ℓ_1 regularizer. These numbers are plotted in Figure 11.10(b).

when $B = 0$, all the coefficients are zero. As we increase B , the coefficients gradually “turn on”.² The analogous result for ridge regression is shown in Figure 11.10(a). For ridge, we see all coefficients are non-zero (assuming $\lambda > 0$), so the solution is not sparse.

Remarkably, it can be shown that the lasso solution path is a piecewise linear function of λ [Efr+04; GL15]. That is, there are a set of critical values of λ where the active set of non-zero coefficients changes. For values of λ between these critical values, each non-zero coefficient increases or decreases in a linear fashion. This is illustrated in Figure 11.10(b). Furthermore, one can solve for these critical values analytically [Efr+04]. In Table 11.1, we display the actual coefficient values at each of these critical steps along the regularization path (the last line is the least squares solution).

By changing λ from $\lambda_{\max}$ to 0, we can go from a solution in which all the weights are zero to a solution in which all weights are non-zero. Unfortunately, not all subset sizes are achievable using lasso. In particular, one can show that, if $D > N$, the optimal solution can have at most N variables in it, before reaching the complete set corresponding to the OLS solution of minimal ℓ_1 norm. In Section 11.4.8, we will see that by using an ℓ_2 regularizer as well as an ℓ_1 regularizer (a method known as the elastic net), we can achieve sparse solutions which contain more variables than training cases. This lets us explore model sizes between N and D .

11.4.5 Comparison of least squares, lasso, ridge and subset selection

In this section, we compare least squares, lasso, ridge and subset selection. For simplicity, we assume all the features of $\mathbf{X}$ are orthonormal, so $\mathbf{X}^\top \mathbf{X} = \mathbf{I}$. In this case, the NLL is given by

$$\text{NLL}(\mathbf{w}) = \|\mathbf{y} - \mathbf{X}\mathbf{w}\|^2 = \mathbf{y}^\top \mathbf{y} + \mathbf{w}^\top \mathbf{X}^\top \mathbf{X} \mathbf{w} - 2\mathbf{w}^\top \mathbf{X}^\top \mathbf{y} \quad (11.91)$$

$$= \text{const} + \sum_d w_d^2 - 2 \sum_d \sum_n w_d x_{nd} y_n \quad (11.92)$$

so we see this factorizes into a sum of terms, one per dimension. Hence we can write down the MAP and ML estimates analytically for each w_d separately, as given below.

- **MLE** From Equation (11.85), the OLS solution is given by

$$\hat{w}_d^{\text{mle}} = c_d / a_d = \mathbf{x}_{:d}^\top \mathbf{y} \quad (11.93)$$

where $\mathbf{x}_{:d}$ is the d 'th column of $\mathbf{X}$.

2. It is common to plot the solution versus the **shrinkage factor**, defined as $s(B) = B/B_{\max}$, rather than against B . This merely affects the scale of the horizontal axis, not the shape of the curves.

Term	OLS	Best Subset	Ridge	Lasso
intercept	2.465	2.477	2.467	2.465
lcalvol	0.676	0.736	0.522	0.548
lweight	0.262	0.315	0.255	0.224
age	-0.141	0.000	-0.089	0.000
lbph	0.209	0.000	0.186	0.129
svi	0.304	0.000	0.259	0.186
lcp	-0.287	0.000	-0.095	0.000
gleason	-0.021	0.000	0.025	0.000
pgg45	0.266	0.000	0.169	0.083
Test error	0.521	0.492	0.487	0.457
Std error	0.176	0.141	0.157	0.146

Figure 11.11: Results of different methods on the prostate cancer data, which has 8 features and 67 training cases. Methods are: OLS = ordinary least squares, Subset = best subset regression, Ridge, Lasso. Rows represent the coefficients; we see that subset regression and lasso give sparse solutions. Bottom row is the mean squared error on the test set (30 cases). Adapted from Table 3.3. of [HTF09]. Generated by [prostate_comparison.ipynb](#).

- **Ridge** One can show that the ridge estimate is given by

$$\hat{w}_d^{\text{ridge}} = \frac{\hat{w}_d^{\text{mle}}}{1 + \lambda} \quad (11.94)$$

- **Lasso** From Equation (11.88), and using the fact that $\hat{w}_d^{\text{mle}} = c_d/a_d$, we have

$$\hat{w}_d^{\text{lasso}} = \text{sign}(\hat{w}_d^{\text{mle}}) (|\hat{w}_d^{\text{mle}}| - \lambda)_+ \quad (11.95)$$

This corresponds to soft thresholding, shown in Figure 11.9(a).

- **Subset selection** If we pick the best K features using subset selection, the parameter estimate is as follows

$$\hat{w}_d^{\text{ss}} = \begin{cases} \hat{w}_d^{\text{mle}} & \text{if } \text{rank}(|\hat{w}_d^{\text{mle}}|) \leq K \\ 0 & \text{otherwise} \end{cases} \quad (11.96)$$

where rank refers to the location in the sorted list of weight magnitudes. This corresponds to hard thresholding, shown in Figure 11.9(b).

We now experimentally compare the prediction performance of these methods on the prostate cancer regression dataset from [HTF09]. (We treat features `gleason` and `svi` as numeric, not categorical.) Figure 11.11 shows the estimated coefficients at the value of λ (or K) chosen by cross-validation; we see that the subset method is the sparsest, then lasso. In terms of predictive performance, all methods are very similar, as can be seen from Figure 11.12.

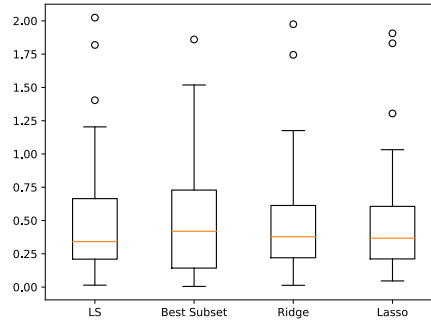


Figure 11.12: Boxplot displaying (absolute value of) prediction errors on the prostate cancer test set for different regression methods. Generated by [prostate_comparison.ipynb](#).

11.4.6 Variable selection consistency

It is common to use ℓ_1 regularization to estimate the set of relevant variables, a process known as **variable selection**. A method that can recover the true set of relevant variables (i.e., the support of $\mathbf{w}^*$) in the $N \rightarrow \infty$ limit is called **model selection consistent**. (This is a theoretical notion that assumes the data comes from the model.)

Let us give an example. We first generate a sparse signal $\mathbf{w}^*$ of size $D = 4096$, consisting of 160 randomly placed ± 1 spikes. Next we generate a random design matrix $\mathbf{X}$ of size $N \times D$, where $N = 1024$. Finally we generate a noisy observation $\mathbf{y} = \mathbf{X}\mathbf{w}^* + \boldsymbol{\epsilon}$, where $\epsilon_n \sim \mathcal{N}(0, 0.01^2)$. We then estimate $\mathbf{w}$ from $\mathbf{y}$ and $\mathbf{X}$. The original $\mathbf{w}^*$ is shown in the first row of Figure 11.13. The second row is the ℓ_1 estimate $\hat{\mathbf{w}}_{L1}$ using $\lambda = 0.1\lambda_{\max}$. We see that this has “spikes” in the right places, so it has correctly identified the relevant variables. However, although we see that $\hat{\mathbf{w}}_{L1}$ has correctly identified the non-zero components, but they are too small, due to shrinkage. In the third row, we show the results of using the debiasing technique discussed in Section 11.4.3. This shows that we can recover the original weight vector. By contrast, the final row shows the OLS estimate, which is dense. Furthermore, it is visually clear that there is no single threshold value we can apply to $\hat{\mathbf{w}}_{\text{mle}}$ to recover the correct sparse weight vector.

To use lasso to perform variable selection, we have to pick λ . It is common to use cross validation to pick the optimal value on the regularization path. However, it is important to note that cross validation is picking a value of λ that results in good predictive accuracy. This is not usually the same value as the one that is likely to recover the “true” model. To see why, recall that ℓ_1 regularization performs selection *and* shrinkage, that is, the chosen coefficients are brought closer to 0. In order to prevent relevant coefficients from being shrunk in this way, cross validation will tend to pick a value of λ that is not too large. Of course, this will result in a less sparse model which contains irrelevant variables (false positives). Indeed, it was proved in [MB06] that the prediction-optimal value of λ does not result in model selection consistency. However, various extensions to the basic method have been devised that are model selection consistent (see e.g., [BG11; HTW15]).

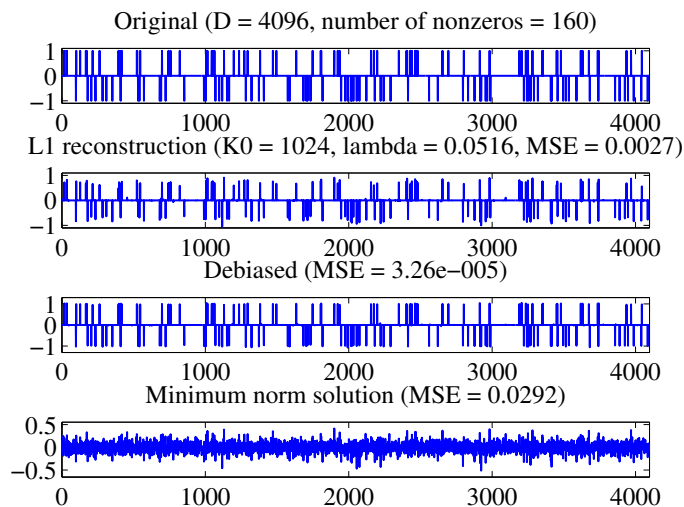


Figure 11.13: Example of recovering a sparse signal using lasso. See text for details. Adapted from Figure 1 of [FNW07]. Generated by `sparse_sensing_demo.ipynb`.

11.4.7 Group lasso

In standard ℓ_1 regularization, we assume that there is a 1:1 correspondence between parameters and variables, so that if $\hat{w}_d = 0$, we interpret this to mean that variable d is excluded. But in more complex models, there may be many parameters associated with a given variable. In particular, each variable d may have a vector of weights $\mathbf{w}_d$ associated with it, so the overall weight vector has block structure, $\mathbf{w} = [\mathbf{w}_1, \mathbf{w}_2, \dots, \mathbf{w}_D]$. If we want to exclude variable d , we have to force the whole subvector $\mathbf{w}_d$ to go to zero. This is called **group sparsity**.

11.4.7.1 Applications

Here are some examples where group sparsity is useful:

- Linear regression with categorical inputs: If the d 'th variable is categorical with K possible levels, then it will be represented as a one-hot vector of length K (Section 1.5.3.1), so to exclude variable d , we have to set the whole vector of incoming weights to 0.
- Multinomial logistic regression: The d 'th variable will be associated with C different weights, one per class (Section 10.3), so to exclude variable d , we have to set the whole vector of outgoing weights to 0.
- Neural networks: the k 'th neuron will have multiple inputs, so if we want to “turn the neuron off”, we have to set all the incoming weights to zero. This allows us to use group sparsity to learn neural network structure (for details, see e.g., [GEH19]).

- Multi-task learning: each input feature is associated with C different weights, one per output task. If we want to use a feature for all of the tasks or none of the tasks, we should select weights at the group level [OTJ07].

11.4.7.2 Penalizing the two-norm

To encourage group sparsity, we partition the parameter vector into G groups, $\mathbf{w} = [\mathbf{w}_1, \dots, \mathbf{w}_G]$. Then we minimize the following objective

$$\text{PNLL}(\mathbf{w}) = \text{NLL}(\mathbf{w}) + \lambda \sum_{g=1}^G \|\mathbf{w}_g\|_2 \quad (11.97)$$

where $\|\mathbf{w}_g\|_2 = \sqrt{\sum_{d \in g} w_d^2}$ is the 2-norm of the group weight vector. If the NLL is least squares, this method is called **group lasso** [YL06; Kyu+10].

Note that if we had used the sum of the squared 2-norms in Equation (11.97), then the model would become equivalent to ridge regression, since

$$\sum_{g=1}^G \|\mathbf{w}_g\|_2^2 = \sum_g \sum_{d \in g} w_d^2 = \|\mathbf{w}\|_2^2 \quad (11.98)$$

By using the square root, we are penalizing the radius of a ball containing the group's weight vector: the only way for the radius to be small is if all elements are small.

Another way to see why the square root version enforces sparsity at the group level is to consider the gradient of the objective. Suppose there is only one group of two variables, so the penalty has the form $\sqrt{w_1^2 + w_2^2}$. The derivative wrt w_1 is

$$\frac{\partial}{\partial w_1} (w_1^2 + w_2^2)^{\frac{1}{2}} = \frac{w_1}{\sqrt{w_1^2 + w_2^2}} \quad (11.99)$$

If w_2 is close to zero, then the derivative approaches 1, and w_1 is driven to zero as well, with force proportional to λ . If, however, w_2 is large, the derivative approaches 0, and w_1 is free to stay large as well. So all the coefficients in the group will have similar size.

11.4.7.3 Penalizing the infinity norm

A variant of this technique replaces the 2-norm with the infinity-norm [TVW05; ZRY05]:

$$\|\mathbf{w}_g\|_\infty = \max_{d \in g} |w_d| \quad (11.100)$$

It is clear that this will also result in group sparsity, since if the largest element in the group is forced to zero, all the smaller ones will be as well.

11.4.7.4 Example

An illustration of these techniques is shown in Figure 11.14 and Figure 11.15. We have a true signal $\mathbf{w}$ of size $D = 2^{12} = 4096$, divided into 64 groups each of size 64. We randomly choose 8 groups

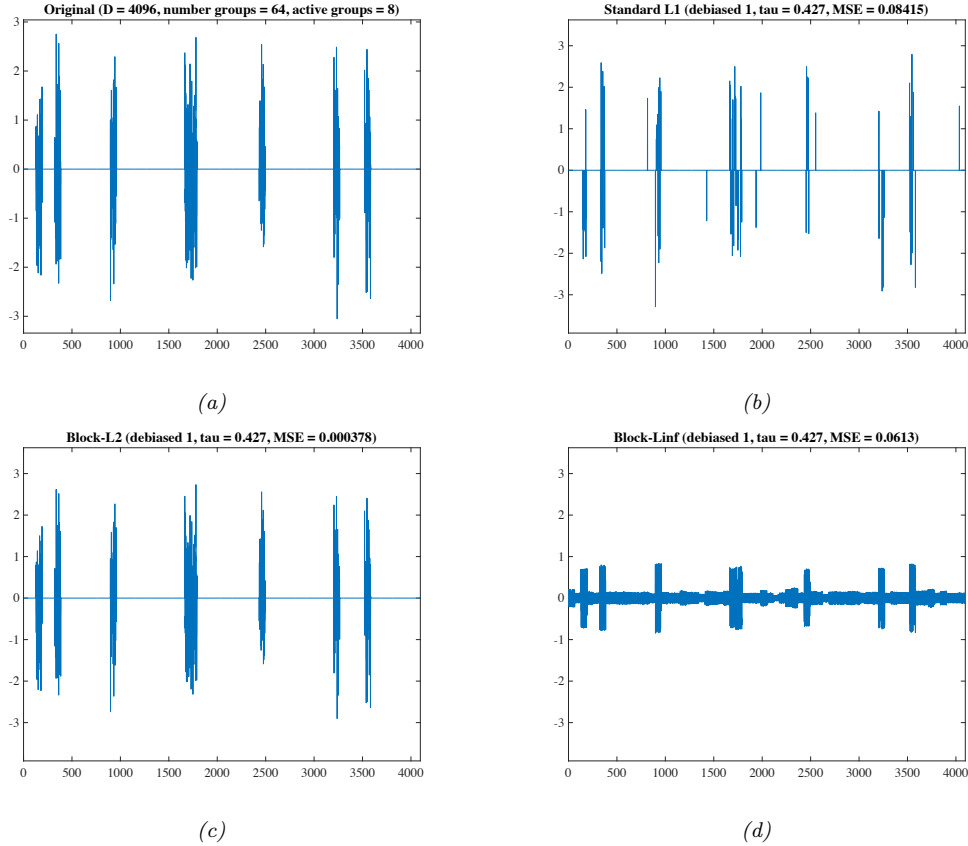


Figure 11.14: Illustration of group lasso where the original signal is piecewise Gaussian. (a) Original signal. (b) Vanilla lasso estimate. (c) Group lasso estimate using an ℓ_2 norm on the blocks. (d) Group lasso estimate using an ℓ_∞ norm on the blocks. Adapted from Figures 3-4 of [WNF09]. Generated by [groupLassoDemo.ipynb](#).

of $\mathbf{w}$ and assign them non-zero values. In Figure 11.14 the values are drawn from a $\mathcal{N}(0, 1)$; in Figure 11.15, the values are all set to 1. We then sample a random design matrix $\mathbf{X}$ of size $N \times D$, where $N = 2^{10} = 1024$. Finally, we generate $\mathbf{y} = \mathbf{X}\mathbf{w} + \boldsymbol{\epsilon}$, where $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, 10^{-4}\mathbf{I}_N)$. Given this data, we estimate the support of $\mathbf{w}$ using ℓ_1 or group ℓ_1 , and then estimate the non-zero values using least squares (debiased estimate).

We see from the figures that group lasso does a much better job than vanilla lasso, since it respects the known group structure. We also see that the ℓ_∞ norm has a tendency to make all the elements within a block to have similar magnitude. This is appropriate in the second example, but not the first. (The value of λ was the same in all examples, and was chosen by hand.)

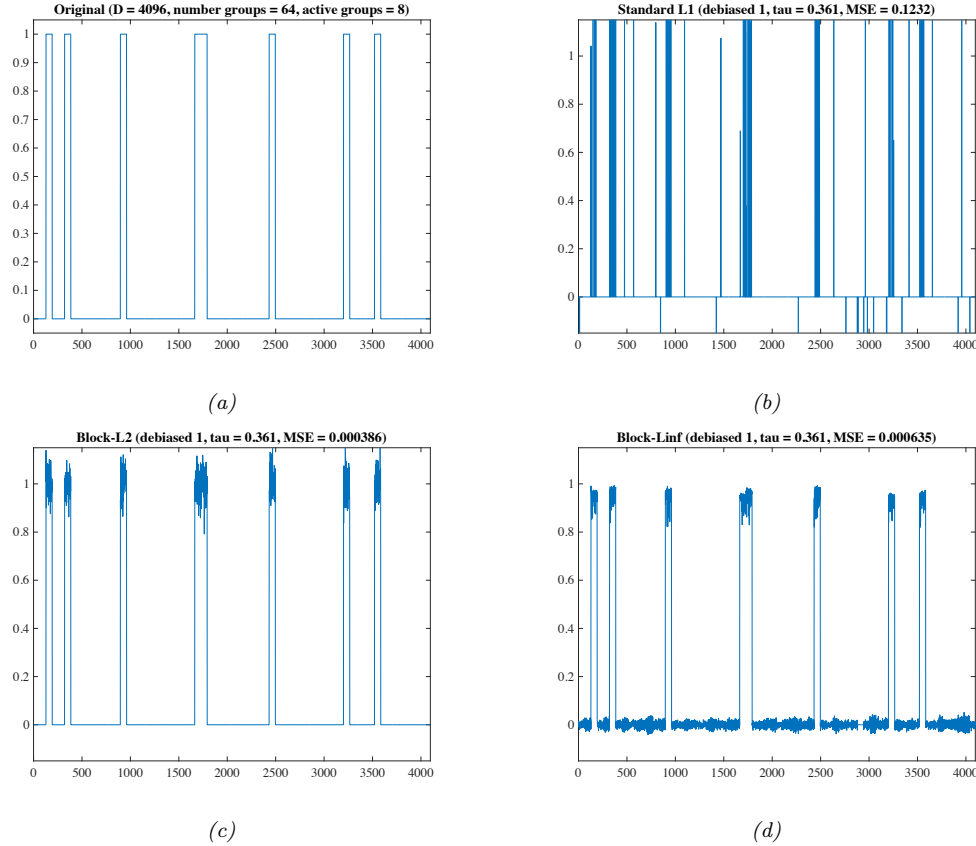


Figure 11.15: Same as Figure 11.14, except the original signal is piecewise constant. Generated by [groupLassoDemo.ipynb](#).

11.4.8 Elastic net (ridge and lasso combined)

In group lasso, we need to specify the group structure ahead of time. For some problems, we don't know the group structure, and yet we would still like highly correlated coefficients to be treated as an implicit group. One way to achieve this effect, proposed in [ZH05], is to use the **elastic net**, which is a hybrid between lasso and ridge regression.³ This corresponds to minimizing the following objective:

$$\mathcal{L}(\mathbf{w}, \lambda_1, \lambda_2) = \|\mathbf{y} - \mathbf{X}\mathbf{w}\|^2 + \lambda_2 \|\mathbf{w}\|_2^2 + \lambda_1 \|\mathbf{w}\|_1 \quad (11.101)$$

This penalty function is *strictly convex* (assuming $\lambda_2 > 0$) so there is a unique global minimum, even if $\mathbf{X}$ is not full rank. It can be shown [ZH05] that any strictly convex penalty on $\mathbf{w}$ will exhibit a **grouping effect**, which means that the regression coefficients of highly correlated variables tend to

3. It is apparently called the “elastic net” because it is “like a stretchable fishing net that retains all the big fish” [ZH05].

be equal. In particular, if two features are identically equal, so $\mathbf{X}_{:j} = \mathbf{X}_{:k}$, one can show that their estimates are also equal, $\hat{w}_j = \hat{w}_k$. By contrast, with lasso, we may have that $\hat{w}_j = 0$ and $\hat{w}_k \neq 0$ or vice versa, resulting in less stable estimates.

In addition to its soft grouping behavior, elastic net has other advantages. In particular, if $D > N$, the maximum number of non-zero elements that can be selected (excluding the MLE, which has D non-zero elements) is N . By contrast, elastic net can select more than N non-zero variables on its path to the dense estimate, thus exploring more possible subsets of variables.

11.4.9 Optimization algorithms

A large variety of algorithms have been proposed to solve the lasso problem, and other ℓ_1 -regularized convex objectives. In this section, we briefly mention some of the most popular methods.

11.4.9.1 Coordinate descent

Sometimes it is hard to optimize all the variables simultaneously, but it is easy to optimize them one by one. In particular, we can solve for the j 'th coefficient with all the others held fixed as follows:

$$w_j^* = \underset{\eta}{\operatorname{argmin}} \mathcal{L}(\mathbf{w} + \eta \mathbf{e}_j) \quad (11.102)$$

where $\mathbf{e}_j$ is the j 'th unit vector. This is called **coordinate descent**. We can either cycle through the coordinates in a deterministic fashion, or we can sample them at random, or we can choose to update the coordinate for which the gradient is steepest.

This method is particularly appealing if each one-dimensional optimization problem can be solved analytically, as is the case for lasso (see Equation (11.87)). This is known as the **shooting** algorithm [Fu98; WL08]. (The term “shooting” is a reference to cowboy theme inspired by the term “lasso”.) See Algorithm 11.1 for details.

This coordinate descent method has been generalized to the GLM case in [FHT10], and is the basis of the popular **glmnet** software library.

Algorithm 11.1: Coordinate descent for lasso (aka shooting algorithm)

```

1 Initialize  $\mathbf{w} = (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^\top \mathbf{y}$ 
2 repeat
3   for  $d = 1, \dots, D$  do
4      $a_d = \sum_{n=1}^N x_{nd}^2$ 
5      $c_d = \sum_{n=1}^N x_{nd}(y_n - \mathbf{w}^\top \mathbf{x}_n + w_d x_{nd})$ 
6      $w_d = \text{SoftThreshold}(\frac{c_d}{a_d}, \lambda/a_d)$ 
7 until converged
```

11.4.9.2 Projected gradient descent

In this section, we convert the non-differentiable ℓ_1 penalty into a smooth regularizer. To do this, we first use the **split variable trick** to define $\mathbf{w} = \mathbf{w}^+ - \mathbf{w}^-$, where $\mathbf{w}^+ = \max\{\mathbf{w}, 0\}$ and

$\mathbf{w}^- = -\min\{\mathbf{w}, 0\}$. Now we can replace $\|\mathbf{w}\|_1$ with $\sum_d (w_d^+ + w_d^-)$. We also have to replace $\text{NLL}(\mathbf{w})$ with $\text{NLL}(\mathbf{w}^+ + \mathbf{w}^-)$. Thus we get the following smooth, but constrained, optimization problem:

$$\min_{\mathbf{w}^+ \geq 0, \mathbf{w}^- \geq 0} \text{NLL}(\mathbf{w}^+ - \mathbf{w}^-) + \lambda \sum_{d=1}^D (w_d^+ + w_d^-) \quad (11.103)$$

In this case of a Gaussian likelihood, the NLL becomes a least squares loss, and the objective becomes a quadratic program (Section 8.5.4). One way to solve such problems is to use projected gradient descent (Section 8.6.1). Specifically, we can enforce the constraint by projecting onto the positive orthant, which we can do using $w_d := \max(w_d, 0)$; this operation is denoted by P_+ . Thus the projected gradient update takes the following form:

$$\begin{pmatrix} \mathbf{w}_{t+1}^+ \\ \mathbf{w}_{t+1}^- \end{pmatrix} = P_+ \left(\begin{pmatrix} \mathbf{w}_t^+ - \eta_t \nabla \text{NLL}(\mathbf{w}_t^+ - \mathbf{w}_t^-) - \eta_t \lambda \mathbf{e} \\ \mathbf{w}_t^- + \eta_t \nabla \text{NLL}(\mathbf{w}_t^+ - \mathbf{w}_t^-) - \eta_t \lambda \mathbf{e} \end{pmatrix} \right) \quad (11.104)$$

where $\mathbf{e}$ is the unit vector of all ones.

11.4.9.3 Proximal gradient descent

In Section 8.6, we introduced proximal gradient descent, which can be used to optimize smooth functions with non-smooth penalties, such as ℓ_1 . In Section 8.6.2, we showed that the proximal operator for the ℓ_1 penalty corresponds to soft thresholding. Thus the proximal gradient descent update can be written as

$$\mathbf{w}_{t+1} = \text{SoftThreshold}(\mathbf{w}_t - \eta_t \nabla \text{NLL}(\mathbf{w}_t), \eta_t \lambda) \quad (11.105)$$

where the soft thresholding operator (Equation (8.134)) is applied elementwise. This is called the **iterative soft thresholding algorithm** or **ISTA** [DDDM04; Don95]. If we combine this with Nesterov acceleration, we get the method known as “fast ISTA” or **FISTA** [BT09], which is widely used to fit sparse linear models.

11.4.9.4 LARS

In this section, we discuss methods that can generate a set of solutions for different values of λ , starting with the empty set, i.e., they compute the full regularization path (Section 11.4.4). These algorithms exploit the fact that one can quickly compute $\hat{\mathbf{w}}(\lambda_k)$ from $\hat{\mathbf{w}}(\lambda_{k-1})$ if $\lambda_k \approx \lambda_{k-1}$; this is known as **warm starting**. In fact, even if we only want the solution for a single value of λ , call it λ_* , it can sometimes be computationally more efficient to compute a set of solutions, from $\lambda_{\max}$ down to λ_* , using warm-starting; this is called a **continuation method** or **homotopy** method. This is often much faster than directly “cold-starting” at λ_* ; this is particularly true if λ_* is small.

The **LARS** algorithm [Efr+04], which stands for “least angle regression and shrinkage”, is an example of a homotopy method for the lasso problem. This can compute $\hat{\mathbf{w}}(\lambda)$ for all possible values of λ in an efficient manner. (A similar algorithm was independently invented in [OPT00b; OPT00a]).

LARS works as follows. It starts with a large value of λ , such that only the variable that is most correlated with the response vector $\mathbf{y}$ is chosen. Then λ is decreased until a second variable is found which has the same correlation (in terms of magnitude) with the current residual as the first variable,

where the residual at step k on the path is defined as $\mathbf{r}_k = \mathbf{y} - \mathbf{X}_{:,F_k} \mathbf{w}_k$, where F_k is the current **active set** (cf., Equation (11.83)). Remarkably, one can solve for this new value of λ analytically, by using a geometric argument (hence the term “least angle”). This allows the algorithm to quickly “jump” to the next point on the regularization path where the active set changes. This repeats until all the variables are added.

It is necessary to allow variables to be removed from the current active set, even as we increase λ , if we want the sequence of solutions to correspond to the regularization path of lasso. If we disallow variable removal, we get a slightly different algorithm called **least angle regression** or **LAR**. LAR is very similar to **greedy forward selection**, and a method known as **least squares boosting** (see e.g., [HTW15]).

11.5 Regression splines *

We have seen how we can use polynomial basis functions to create nonlinear mappings from input to output, even though the model remains linear in the parameters. One problem with polynomials is that they are a global approximation to the function. We can achieve more flexibility by using a series of local approximations. To do this, we just need to define a set of basis functions that have local support. The notion of “locality” is hard to define in high-dimensional input spaces, so in this section, we restrict ourselves to 1d inputs. We can then approximate the function using

$$f(x; \boldsymbol{\theta}) = \sum_{i=1}^m w_i B_i(x) \quad (11.106)$$

where B_i is the i ’th basis function.

A common way to define such basis functions is to use **B-splines**. (“B” stands for “basis”, and the term “spline” refers to a flexible piece of material used by artists to draw curves.) We discuss this in more detail in Section 11.5.1.

11.5.1 B-spline basis functions

A spline is a piecewise polynomial of degree D , where the locations of the pieces are defined by a set of **knots**, $t_1 < \dots < t_m$. More precisely, the polynomial is defined on each of the intervals $(-\infty, t_1)$, $[t_1, t_2]$, $\dots$, $[t_m, \infty)$. The function is continuous and has continuous derivatives of orders $1, \dots, D-1$ at its knot points. It is common to use **cubic splines**, in which $D = 3$. This ensures the function is continuous, and has continuous first and second derivatives at each knot.

We will skip the details on how B-splines are computed, since it is not relevant to our purposes. Suffice it to say that we can call the `patsy.bs` function to convert the $N \times 1$ data matrix $\mathbf{X}$ into an $N \times (K + D + 1)$ design matrix $\mathbf{B}$, where K is the number of knots and D is the degree. (Alternatively, you can specify the desired number of basis functions, and let `patsy` work out the number and locations of the knots.)

Figure 11.16 illustrates this approach, where we use B-splines of degree 0, 1 and 3, with 3 knots. By taking a weighted combination of these basis functions, we can get increasingly smooth functions, as shown in the bottom row.

We see from Figure 11.16 that each individual basis function has local support. At any given input point x , only $D + 1$ basis functions will be “active”. This is more obvious if we plot the design matrix

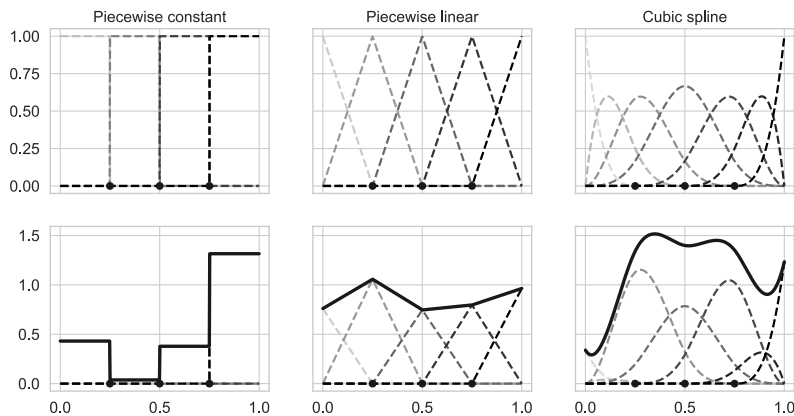


Figure 11.16: Illustration of B-splines of degree 0, 1 and 3. Top row: unweighted basis functions. Dots mark the locations of the 3 internal knots at $[0.25, 0.5, 0.75]$. Bottom row: weighted combination of basis functions using random weights. Generated by `splines_basis_weighted.ipynb`. Adapted from Figure 5.4 of [MKL11]. Used with kind permission of Osvaldo Martin.

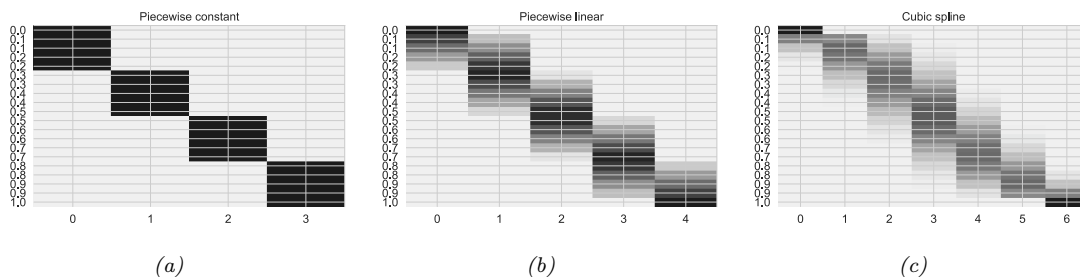


Figure 11.17: Design matrix for B-splines of degree (a) 0, (b) 1 and (c) 3. We evaluate the splines on 20 inputs ranging from 0 to 1. Generated by `splines_basis_heatmap.ipynb`. Adapted from Figure 5.6 of [MKL11]. Used with kind permission of Osvaldo Martin.

B itself. Let us first consider the piecewise constant spline, shown in Figure 11.17(a). The first B-spline (column 1) is 1 for the first 5 observations, and otherwise 0. The second B-spline (column 0) is 0 for the first 5 observations, 1 for the second 5, and then 0 again. And so on. Now consider the linear spline, shown in Figure 11.17(b). The first B-spline (column 0) goes from 1 to 0, the next three splines go from 0 to 1 and back to 0; and the last spline (column 4) goes from 0 to 1; this reflects the triangular shapes shown in the top middle panel of Figure 11.16. Finally consider the cubic spline, shown in Figure 11.17(c). Here the pattern of activations is smoother, and the resulting model fits will be smoother too.

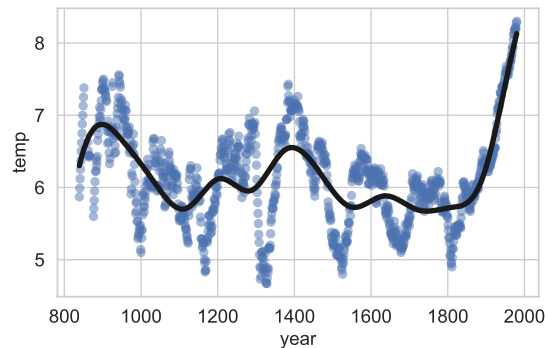


Figure 11.18: Fitting a cubic spline regression model with 15 knots to a 1d dataset. Generated by `splines_cherry_blossoms.ipynb`. Adapted from Figure 5.3 of [McE20].

11.5.2 Fitting a linear model using a spline basis

Once we have computed the design matrix $\mathbf{B}$, we can use it to fit a linear model using least squares or ridge regression. (It is usually best to use some regularization.) As an example, we consider a dataset from [McE20, Sec 4.5], which records the the first day of the year, and the corresponding temperature, that marks the start of the cherry blossom season in Japan. (We use this dataset since it has interesting semi-periodic structure.) We fit the data using a cubic spline. We pick 15 knots, spaced according to quantiles of the data. The results are shown in Figure 11.18. We see that the fit is reasonable. Using more knots would improve the quality of the fit, but would eventually result in overfitting. We can select the number of knots using a model selection method, such as grid search plus cross validation.

11.5.3 Smoothing splines

Smoothing splines are related to regression splines, but use N knots, where N is the number of datapoints. That is, they are non-parametric models, since the number of parameters grows with the size of the data, rather than being fixed a priori. To avoid overfitting, smoothing splines rely on ℓ_2 regularization. This technique is closely related to Gaussian process regression, which we discuss in Section 17.2.

11.5.4 Generalized additive models

A **generalized additive model** or **GAM** extends spline regression to the case of multidimensional inputs [HT90]. It does this by ignoring interactions between the inputs, and assuming the function has the following additive form:

$$f(\mathbf{x}; \boldsymbol{\theta}) = \alpha + \sum_{d=1}^D f_d(x_d) \quad (11.107)$$

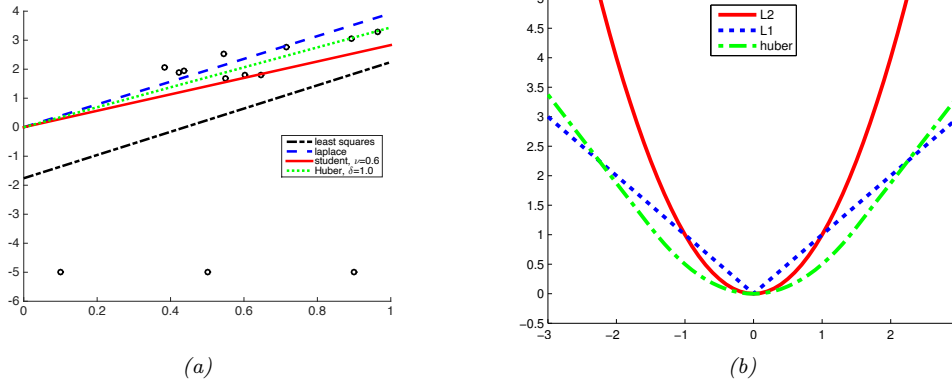


Figure 11.19: (a) Illustration of robust linear regression. Generated by [linregRobustDemoCombined.ipynb](#). (b) Illustration of ℓ_2 , ℓ_1 , and Huber loss functions with $\delta = 1.5$. Generated by [huberLossPlot.ipynb](#).

where each f_d is a regression or smoothing spline. This model can be fit using **backfitting**, which iteratively fits each f_d to the partial residuals generated by the other terms. We can extend GAMs beyond the regression case (e.g., to classification) by using a link function, as in generalized linear models (Chapter 12).

11.6 Robust linear regression *

It is very common to model the noise in regression models using a Gaussian distribution with zero mean and constant variance, $r_n \sim \mathcal{N}(0, \sigma^2)$, where $r_n = y_n - \mathbf{w}^\top \mathbf{x}_n$. In this case, maximizing likelihood is equivalent to minimizing the sum of squared residuals, as we have seen. However, if we have **outliers** in our data, this can result in a poor fit, as illustrated in Figure 11.19(a). (The outliers are the points on the bottom of the figure.) This is because squared error penalizes deviations quadratically, so points far from the line have more effect on the fit than points near to the line.

One way to achieve **robustness** to outliers is to replace the Gaussian distribution for the response variable with a distribution that has **heavy tails**. Such a distribution will assign higher likelihood to outliers, without having to perturb the straight line to “explain” them. We discuss several possible alternative probability distributions for the response variable below; see Table 11.2 for a summary.

11.6.1 Laplace likelihood

In Section 2.7.3, we noted that the Laplace distribution is also robust to outliers. If we use this as our observation model for regression, we get the following likelihood:

$$p(y|\mathbf{x}, \mathbf{w}, b) = \text{Laplace}(y|\mathbf{w}^\top \mathbf{x}, b) \propto \exp\left(-\frac{1}{b}|y - \mathbf{w}^\top \mathbf{x}|\right) \quad (11.108)$$

The robustness arises from the use of $|y - \mathbf{w}^\top \mathbf{x}|$ instead of $(y - \mathbf{w}^\top \mathbf{x})^2$. Figure 11.19(a) gives an example of the method in action.

Likelihood	Prior	Posterior	Name	Section
Gaussian	Uniform	Point	Least squares	11.2.2
Student	Uniform	Point	Robust regression	11.6.2
Laplace	Uniform	Point	Robust regression	11.6.1
Gaussian	Gaussian	Point	Ridge	11.3
Gaussian	Laplace	Point	Lasso	11.4
Gaussian	Gauss-Gamma	Gauss-Gamma	Bayesian lin. reg	11.7

Table 11.2: Summary of various likelihoods, priors and posteriors used for linear regression. The likelihood refers to the distributional form of $p(y|\mathbf{x}, \mathbf{w}, \sigma^2)$, and the prior refers to the distributional form of $p(\mathbf{w})$. The posterior refers to the distributional form of $p(\mathbf{w}|\mathcal{D})$. “Point” stands for the degenerate distribution $\delta(\mathbf{w} - \hat{\mathbf{w}})$, where $\hat{\mathbf{w}}$ is the MAP estimate. MLE is equivalent to using a point posterior and a uniform prior.

11.6.1.1 Computing the MLE using linear programming

We can compute the MLE for this model using linear programming. As we explain in Section 8.5.3, this is a way to solve a constrained optimization problems of the form

$$\underset{\mathbf{v}}{\operatorname{argmin}} \mathbf{c}^\top \mathbf{v} \quad \text{s.t.} \quad \mathbf{A}\mathbf{v} \leq \mathbf{b} \quad (11.109)$$

where $\mathbf{v} \in \mathbb{R}^n$ is the set of n unknown parameters, $\mathbf{c}^\top \mathbf{v}$ is the linear objective function we want to minimize, and $\mathbf{a}_i^\top \mathbf{v} \leq b_i$ is a set of m linear constraints we must satisfy. To apply this to our problem, let us define $\mathbf{v} = (w_1, \dots, w_D, e_1, \dots, e_N) \in \mathbb{R}^{D+N}$, where $e_i = |y_i - \hat{y}_i|$ is the residual error for example i . We want to minimize the sum of the residuals, so we define $\mathbf{c} = (0, \dots, 0, 1, \dots, 1) \in \mathbb{R}^{D+N}$, where the first D elements are 0, and the last N elements are 1.

We need to enforce the constraint that $e_i = |\hat{y}_i - y_i|$. In fact it is sufficient to enforce the constraint that $|\mathbf{w}^\top \mathbf{x}_i - y_i| \leq e_i$, since minimizing the sum of the e_i ’s will “push down” on this constraint and make it tight. Since $|a| \leq b \implies -b \leq a \leq b$, we can encode $|\mathbf{w}^\top \mathbf{x}_i - y_i| \leq e_i$ as two linear constraints:

$$e_i \geq \mathbf{w}^\top \mathbf{x}_i - y_i \quad (11.110)$$

$$e_i \geq -(\mathbf{w}^\top \mathbf{x}_i - y_i) \quad (11.111)$$

We can write Equation (11.110) as

$$(\mathbf{x}_i, 0, \dots, 0, -1, 0, \dots, 0)^\top \mathbf{v} \leq y_i \quad (11.112)$$

where the first D entries are filled with $\mathbf{x}_i$, and the -1 is in the $(D + i)$ ’th entry of the vector. Similarly we can write Equation (11.111) as

$$(-\mathbf{x}_i, 0, \dots, 0, -1, 0, \dots, 0)^\top \mathbf{v} \leq -y_i \quad (11.113)$$

We can write these constraints in the form $\mathbf{A}\mathbf{v} \leq \mathbf{b}$ by defining $\mathbf{A} \in \mathbb{R}^{2N \times (N+D)}$ as follows:

$$\mathbf{A} = \begin{pmatrix} \mathbf{x}_1 & -1 & 0 & 0 \cdots & 0 \\ -\mathbf{x}_1 & -1 & 0 & 0 \cdots & 0 \\ \mathbf{x}_2 & 0 & -1 & 0 \cdots & 0 \\ -\mathbf{x}_2 & 0 & -1 & 0 \cdots & 0 \\ \vdots & & & & \end{pmatrix} \quad (11.114)$$

and defining $\mathbf{b} \in \mathbb{R}^{2N}$ as

$$\mathbf{b} = (y_1, -y_1, y_2, -y_2, \dots, y_N, -y_N) \quad (11.115)$$

11.6.2 Student- t likelihood

In Section 2.7.1, we discussed the robustness properties of the Student distribution. To use this in a regression context, we can just make the mean be a linear function of the inputs, as proposed in [Zel76]:

$$p(y|\mathbf{x}, \mathbf{w}, \sigma^2, \nu) = \mathcal{T}(y|\mathbf{w}^\top \mathbf{x}, \sigma^2, \nu) \quad (11.116)$$

We can fit this model using SGD or EM (see [Mur23] for details).

11.6.3 Huber loss

An alternative to minimizing the NLL using a Laplace or Student likelihood is to use the **Huber loss**, which is defined as follows:

$$\ell_{\text{huber}}(r, \delta) = \begin{cases} r^2/2 & \text{if } |r| \leq \delta \\ \delta|r| - \delta^2/2 & \text{if } |r| > \delta \end{cases} \quad (11.117)$$

This is equivalent to ℓ_2 for errors that are smaller than δ , and is equivalent to ℓ_1 for larger errors. See Figure 5.3 for a plot.

The advantage of this loss function is that it is everywhere differentiable. Consequently optimizing the Huber loss is much faster than using the Laplace likelihood, since we can use standard smooth optimization methods (such as SGD) instead of linear programming. Figure 11.19 gives an illustration of the Huber loss function in action. The results are qualitatively similar to the Laplace and Student methods.

The parameter δ , which controls the degree of robustness, is usually set by hand, or by cross-validation. However, [Bar19] shows how to approximate the Huber loss such that we can optimize δ by gradient methods.

11.6.4 RANSAC

In the computer vision community, a common approach to robust regression is to use **RANSAC**, which stands for “random sample consensus” [FB81]. This works as follows: we sample a small initial set of points, fit the model to them, identify outliers wrt this model (based on large residuals), remove

the outliers, and then refit the model to the inliers. We repeat this for many random initial sets and pick the best model.

A deterministic alternative to RANSAC is the following iterative scheme: initially we assume that all datapoints are inliers, and we fit the model to compute $\hat{\mathbf{w}}_0$; then, for each iteration t , we identify the outlier points as those with large residual under the model $\hat{\mathbf{w}}_t$, remove them, and refit the model to the remaining points to get $\hat{\mathbf{w}}_{t+1}$. Even though this hard thresholding scheme makes the problem nonconvex, this simple scheme can be proved to rapidly converge to the optimal estimate under some reasonable assumptions [Muk+19; Sug+19].

11.7 Bayesian linear regression *

We have seen how to compute the MLE and MAP estimate for linear regression models under various priors. In this section, we discuss how to compute the posterior over the parameters, $p(\boldsymbol{\theta}|\mathcal{D})$. For simplicity, we assume the variance is known, so we just want to compute $p(\mathbf{w}|\mathcal{D}, \sigma^2)$. See the sequel to this book, [Mur23], for the general case.

11.7.1 Priors

For simplicity, we will use a Gaussian prior:

$$p(\mathbf{w}) = \mathcal{N}(\mathbf{w} | \check{\mathbf{w}}, \check{\boldsymbol{\Sigma}}) \quad (11.118)$$

This is a small generalization of the prior that we use in ridge regression (Section 11.3). See the sequel to this book, [Mur23], for a discussion of other priors.

11.7.2 Posteriors

We can rewrite the likelihood in terms of an MVN as follows:

$$p(\mathcal{D}|\mathbf{w}, \sigma^2) = \prod_{n=1}^N p(y_n|\mathbf{w}^\top \mathbf{x}, \sigma^2) = \mathcal{N}(\mathbf{y}|\mathbf{X}\mathbf{w}, \sigma^2 \mathbf{I}_N) \quad (11.119)$$

where $\mathbf{I}_N$ is the $N \times N$ identity matrix. We can then use Bayes rule for Gaussians (Equation (3.37)) to derive the posterior, which is as follows:

$$p(\mathbf{w}|\mathbf{X}, \mathbf{y}, \sigma^2) \propto \mathcal{N}(\mathbf{w} | \check{\mathbf{w}}, \check{\boldsymbol{\Sigma}}) \mathcal{N}(\mathbf{y}|\mathbf{X}\mathbf{w}, \sigma^2 \mathbf{I}_N) = \mathcal{N}(\mathbf{w} | \hat{\mathbf{w}}, \hat{\boldsymbol{\Sigma}}) \quad (11.120)$$

$$\hat{\mathbf{w}} \triangleq \hat{\boldsymbol{\Sigma}} (\check{\boldsymbol{\Sigma}}^{-1} \check{\mathbf{w}} + \frac{1}{\sigma^2} \mathbf{X}^\top \mathbf{y}) \quad (11.121)$$

$$\hat{\boldsymbol{\Sigma}} \triangleq (\check{\boldsymbol{\Sigma}}^{-1} + \frac{1}{\sigma^2} \mathbf{X}^\top \mathbf{X})^{-1} \quad (11.122)$$

where $\hat{\mathbf{w}}$ is the posterior mean, and $\hat{\boldsymbol{\Sigma}}$ is the posterior covariance.

If $\check{\mathbf{w}} = \mathbf{0}$ and $\check{\boldsymbol{\Sigma}} = \tau^2 \mathbf{I}$, then the posterior mean becomes $\hat{\mathbf{w}} = \frac{1}{\sigma^2} \hat{\boldsymbol{\Sigma}} \mathbf{X}^\top \mathbf{y}$. If we define $\lambda = \frac{\sigma^2}{\tau^2}$, we recover the ridge regression estimate, $\hat{\mathbf{w}} = (\lambda \mathbf{I} + \mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$, which matches Equation (11.57).

11.7.3 Example

Suppose we have a 1d regression model of the form $f(x; \mathbf{w}) = w_0 + w_1 x_1$, where the true parameters are $w_0 = -0.3$ and $w_1 = 0.5$. We now perform inference $p(\mathbf{w}|\mathcal{D})$ and visualize the 2d prior and posterior as the size of the training set N increases.

In particular, in Figure 11.20 (which inspired the front cover of this book), we plot the likelihood, the posterior, and an approximation to the posterior predictive distribution.⁴ Each row plots these distributions as we increase the amount of training data, N . We now explain each row:

- In the first row, $N = 0$, so the posterior is the same as the prior. In this case, our predictions are “all over the place”, since our prior is essentially uniform.
- In the second row, $N = 1$, so we have seen one data point (the blue circle in the plot in the third column). Our posterior becomes constrained by the corresponding likelihood, and our predictions pass close to the observed data. However, we see that the posterior has a ridge-like shape, reflecting the fact that there are many possible solutions, with different slopes/intercepts. This makes sense since we cannot uniquely infer two parameters (w_0 and w_1) from one observation.
- In the third row, $N = 2$. In this case, the posterior becomes much narrower since we have two constraints from the likelihood. Our predictions about the future are all now closer to the training data.
- In the fourth (last) row, $N = 100$. Now the posterior is essentially a delta function, centered on the true value of $\mathbf{w}_* = (-0.3, 0.5)$, indicated by a white cross in the plots in the first and second columns. The variation in our predictions is due to the inherent Gaussian noise with magnitude σ^2 .

This example illustrates that, as the amount of data increases, the posterior mean estimate, $\hat{\boldsymbol{\mu}} = \mathbb{E}[\mathbf{w}|\mathcal{D}]$, converges to the true value $\mathbf{w}_*$ that generated the data. We thus say that the Bayesian estimate is a consistent estimator (see Section 5.3.2 for more details). We also see that our posterior uncertainty decreases over time. This is what we mean when we say we are “learning” about the parameters as we see more data.

11.7.4 Computing the posterior predictive

We have discussed how to compute our uncertainty about the parameters of the model, $p(\mathbf{w}|\mathcal{D})$. But what about the uncertainty associated with our predictions about future outputs? Using Equation (3.38), we can show that the posterior predictive distribution at a test point $\mathbf{x}$ is also Gaussian:

$$p(y|\mathbf{x}, \mathcal{D}, \sigma^2) = \int \mathcal{N}(y|\mathbf{x}^\top \mathbf{w}, \sigma^2) \mathcal{N}(\mathbf{w} | \hat{\boldsymbol{\mu}}, \hat{\boldsymbol{\Sigma}}) d\mathbf{w} \quad (11.123)$$

$$= \mathcal{N}(y | \hat{\boldsymbol{\mu}}^\top \mathbf{x}, \hat{\sigma}^2(\mathbf{x})) \quad (11.124)$$

4. To approximate this, we draw some samples from the posterior, $\mathbf{w}_s \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$, and then plot the line $\mathbb{E}[y|\mathbf{x}, \mathbf{w}_s]$, where x ranges over $[-1, 1]$, for each sampled parameter value.

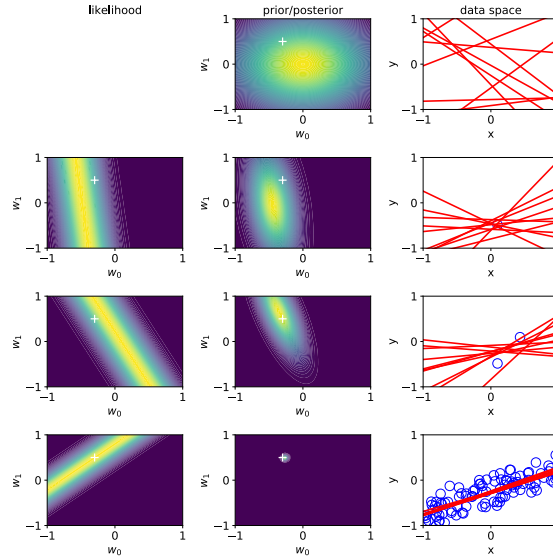


Figure 11.20: Sequential Bayesian inference of the parameters of a linear regression model $p(y|\mathbf{x}) = \mathcal{N}(y|w_0 + w_1x_1, \sigma^2)$. Left column: likelihood function for current data point. Middle column: posterior given first N data points, $p(w_0, w_1|\mathbf{x}_{1:N}, y_{1:N}, \sigma^2)$. Right column: samples from the current posterior predictive distribution. Row 1: prior distribution ($N = 0$). Row 2: after 1 data point. Row 3: after 2 data points. Row 4: after 100 data points. The white cross in columns 1 and 2 represents the true parameter value; we see that the mode of the posterior rapidly converges to this point. The blue circles in column 3 are the observed data points. Adapted from Figure 3.7 of [Bis06]. Generated by `linreg_2d_bayes_demo.ipynb`.

where $\hat{\sigma}^2(\mathbf{x}) \triangleq \sigma^2 + \mathbf{x}^\top \hat{\Sigma} \mathbf{x}$ is the variance of the posterior predictive distribution at point $\mathbf{x}$ after seeing the N training examples. The predicted variance depends on two terms: the variance of the observation noise, σ^2 , and the variance in the parameters, $\hat{\Sigma}$. The latter translates into variance about observations in a way which depends on how close $\mathbf{x}$ is to the training data $\mathcal{D}$. This is illustrated in Figure 11.21(b), where we see that the error bars get larger as we move away from the training points, representing increased uncertainty. This can be important for certain applications, such as active learning, where we choose where to collect training data (see Section 19.4).

In some cases, it is computationally intractable to compute the parameter posterior, $p(\mathbf{w}|\mathcal{D})$. In such cases, we may choose to use a point estimate, $\hat{\mathbf{w}}$, and then to use the plugin approximation. This gives

$$p(y|\mathbf{x}, \mathcal{D}, \sigma^2) = \int \mathcal{N}(y|\mathbf{x}^\top \mathbf{w}, \sigma^2) \delta(\mathbf{w} - \hat{\mathbf{w}}) d\mathbf{w} = p(y|\mathbf{x}^\top \hat{\mathbf{w}}, \sigma^2). \quad (11.125)$$

We see that the posterior predictive variance is constant, and independent of the data, as illustrated in Figure 11.21(a). If we sample a parameter from this posterior, we will always recover a single function, as shown in Figure 11.21(c). By contrast, if we sample from the true posterior, $\mathbf{w}_s \sim p(\mathbf{w}|\mathcal{D}, \sigma^2)$, we will get a range of different functions, as shown in Figure 11.21(d), which more accurately reflects our uncertainty.

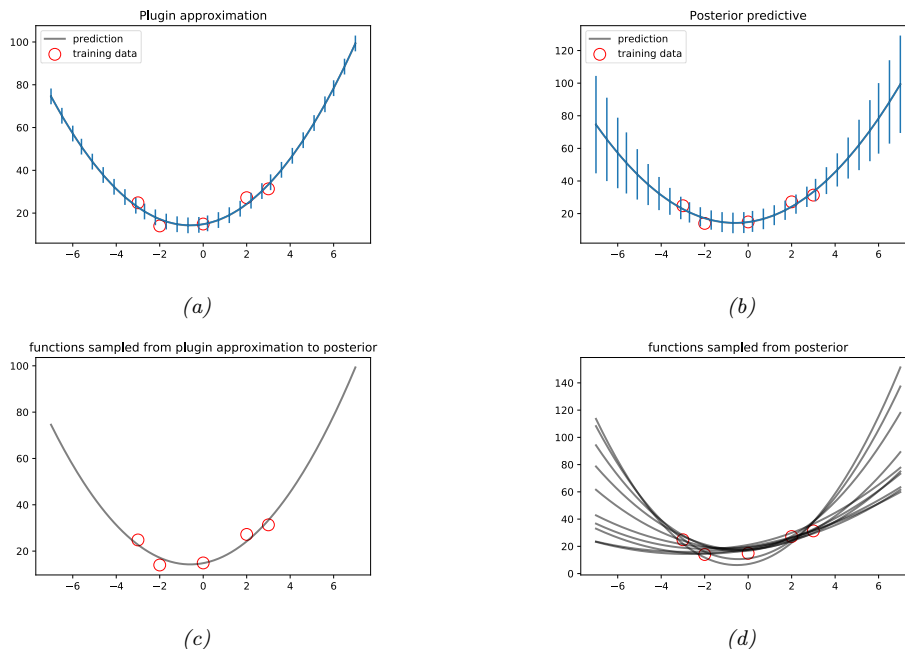


Figure 11.21: (a) Plugin approximation to predictive density (we plug in the MLE of the parameters) when fitting a second degree polynomial to some 1d data. (b) Posterior predictive density, obtained by integrating out the parameters. Black curve is posterior mean, error bars are 2 standard deviations of the posterior predictive density. (c) 10 samples from the plugin approximation to posterior predictive distribution. (d) 10 samples from the true posterior predictive distribution. Generated by [linreg_post_pred_plot.ipynb](#).

11.7.5 The advantage of centering

The astute reader might notice that the shape of the 2d posterior in Figure 11.20 is an elongated ellipse (which eventually collapses to a point as $N \rightarrow \infty$). This implies that there is a lot of posterior correlation between the two parameters, which can cause computational difficulties.

To understand why this happens, note that each data point induces a likelihood function corresponding to a line which goes through that data point. When we look at all the data together, we see that predictions with maximum likelihood must correspond to lines that go through the mean of the data, $(\bar{x}, \bar{y})$. There are many such lines, but if we increase the slope, we must decrease the intercept. Thus we can think of the set of high probability lines as spinning around the data mean, like a wheel of fortune.⁵ This correlation between w_0 and w_1 is why the posterior has the form of a diagonal line. (The Gaussian prior converts this into an elongated ellipse, but the posterior correlation still persists until the sample size causes the posterior to shrink to a point.)

It can be hard to compute such elongated posteriors. One simple solution is to center the input data, i.e., by using $x'_n = x_n - \bar{x}$. Now the lines can pivot around the origin, reducing the posterior

5. This analogy is from [Mar18, p96].

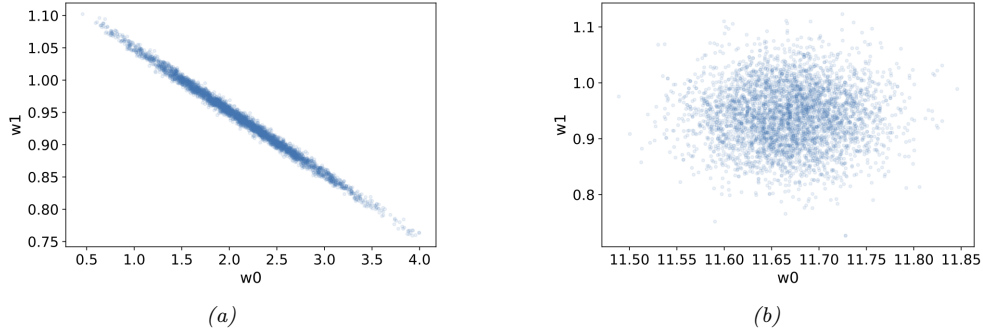


Figure 11.22: Posterior samples of $p(w_0, w_1 | \mathcal{D})$ for 1d linear regression model $p(y|x, \theta) = \mathcal{N}(y|w_0 + w_1x, \sigma^2)$ with a Gaussian prior. (a) Original data. (b) Centered data. Generated by [lin-reg_2d_bayes_centering_pymc3.ipynb](#).

correlation between w_0 and w_1 . See Figure 11.22 for an illustration. (We may also choose to divide each x_n by the standard deviation of that feature, as discussed in Section 10.2.8.)

Note that we can convert the posterior derived from fitting to the centered data back to the original coordinates by noting that

$$y' = w'_0 + w'_1x' = w'_0 + w'_1(x - \bar{x}) = (w'_0 - w'_1\bar{x}) + w'_1x \quad (11.126)$$

Thus the parameters on the uncentered data are $w_0 = w'_0 - w'_1\bar{x}$ and $w_1 = w'_1$.

11.7.6 Dealing with multicollinearity

In many datasets, the input variables can be highly correlated with each other. Including all of them does not generally harm predictive accuracy (provided you use a suitable prior or regularizer to prevent overfitting). However, it can make interpretation of the coefficients more difficult.

To illustrate this, we use a toy example from [McE20, Sec 6.1]. Suppose we have a dataset of N people in which we record their heights h_i , as well as the length of their left legs l_i and right legs r_i . Suppose $h_i \sim \mathcal{N}(10, 2)$, so the average height is $\bar{h} = 10$ (in unspecified units). Suppose the length of the legs is some fraction $\rho_i \sim \text{Unif}(0.4, 0.5)$ of the height, plus a bit of Gaussian noise, specifically $l_i \sim \mathcal{N}(\rho_i h_i, 0.02)$ and $r_i \sim \mathcal{N}(\rho_i h_i, 0.02)$.

Now suppose we want to predict the height of a person given measurement of their leg lengths. (I did mention this is a toy example!) Since both left and right legs are noisy measurements of the unknown quantity, it is useful to use both of them. So we use linear regression to fit $p(h|l, r) = \mathcal{N}(h|\alpha + \beta_l l + \beta_r r, \sigma^2)$. We use vague priors, $\alpha, \beta_l, \beta_r \sim \mathcal{N}(0, 100)$, and $\sigma \sim \text{Expon}(1)$.

Since the average leg length is $\bar{l} = 0.45\bar{h} = 4.5$, we might expect each β coefficient to be around $\bar{h}/\bar{l} = 10/4.5 = 2.2$. However, the posterior marginals shown in Figure 11.23 tell a different story: we see that the posterior mean of β_l is near 2.6, but β_r is near -0.6. Thus it seems like the right leg feature is not needed. This is because the regression coefficient for feature j encodes the value of knowing x_j given that all the other features $\mathbf{x}_{-j}$ are already known, as we discussed in Section 11.2.2.1. If we already know the left leg, the marginal value of also knowing the right leg is small. However, if we

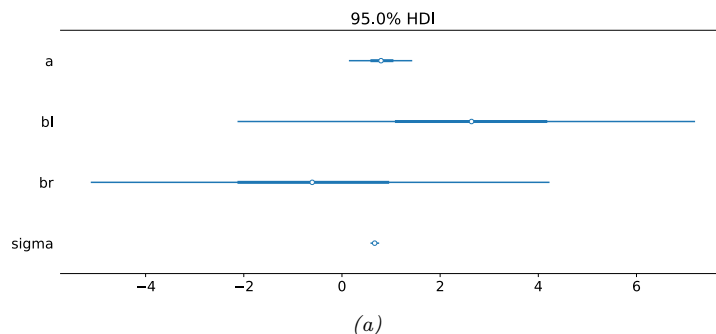


Figure 11.23: Posterior marginals for the parameters in the multi-leg example. Generated by `multi_collinear_legs_numpyro.ipynb`.

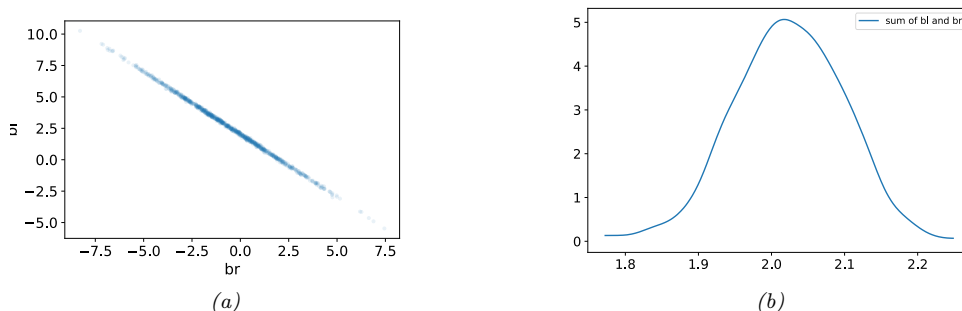


Figure 11.24: Posteriors for the multi-leg example. (a) Joint posterior $p(\beta_l, \beta_r | \mathcal{D})$ (b) Posterior of $p(\beta_l + \beta_r | \text{data})$. Generated by `multi_collinear_legs_numpyro.ipynb`.

rerun this example with slightly different data, we may reach the opposite conclusion, and favor the right leg over the left.

We can gain more insight by looking at the joint distribution $p(\beta_l, \beta_r | \mathcal{D})$, shown in Figure 11.24a. We see that the parameters are very highly correlated, so if β_r is large, then β_l is small, and vice versa. The marginal distribution for each parameter does not capture this. However, it does show that there is a lot of uncertainty about each parameter, showing that they are **non-identifiable**. However, their *sum* is well-determined, as can be seen from Figure 11.24b, where we plot $p(\beta_l + \beta_r | \mathcal{D})$; this is centered on 2.2, as we might expect.

This example goes to show that we must be careful trying to interpret the significance of individual coefficient estimates in a model, since they do not mean much in isolation.

11.7.7 Automatic relevancy determination (ARD) *

Consider a linear regression model with known observation noise but unknown regression weights, $\mathcal{N}(\mathbf{y} | \mathbf{X}\mathbf{w}, \sigma^2 \mathbf{I})$. Suppose we use a Gaussian prior for the weights, $w_j \sim \mathcal{N}(0, 1/\alpha_j)$, where α_j is the

precision of the j 'th parameter. Now suppose we estimate the prior precisions as follows:

$$\hat{\alpha} = \underset{\alpha}{\operatorname{argmax}} p(\mathbf{y}|\mathbf{X}, \alpha) \quad (11.127)$$

where

$$p(\mathbf{y}|\mathbf{X}, \alpha) = \int p(\mathbf{y}|\mathbf{X}\mathbf{w}, \sigma^2) p(\mathbf{w}|\mathbf{0}, \operatorname{diag}(\alpha)^{-1}) d\mathbf{w} \quad (11.128)$$

is the marginal likelihood. This is an example of empirical Bayes, since we are estimating the prior from data. We can view this as a computational shortcut to a fully Bayesian approach. However, there are additional advantages. In particular, suppose, after estimating α , we compute the MAP estimate

$$\hat{\mathbf{w}} = \underset{\mathbf{w}}{\operatorname{argmax}} \mathcal{N}(\mathbf{w}|\mathbf{0}, \hat{\alpha}^{-1}) \quad (11.129)$$

This results in a sparse estimate for $\hat{\mathbf{w}}$, which is perhaps surprising given that the Gaussian prior for $\mathbf{w}$ is not sparsity promoting. The reasons for this are explained in the sequel to this book.

This technique is known as **sparse Bayesian learning** [Tip01] or **automatic relevancy determination (ARD)** [Mac95; Nea96]. It was originally developed for neural networks (where sparsity is applied to the first layer weights), but here we apply it to linear models. See also Section 17.4.1, where we apply it kernelized linear models.

11.8 Exercises

Exercise 11.1 [Multi-output linear regression †]

(Source: Jaakkola.)

Consider a linear regression model with a 2 dimensional response vector $\mathbf{y}_i \in \mathbb{R}^2$. Suppose we have some binary input data, $x_i \in \{0, 1\}$. The training data is as follows:

x	y
0	$(-1, -1)^T$
0	$(-1, -2)^T$
0	$(-2, -1)^T$
1	$(1, 1)^T$
1	$(1, 2)^T$
1	$(2, 1)^T$

Let us embed each x_i into 2d using the following basis function:

$$\phi(0) = (1, 0)^T, \quad \phi(1) = (0, 1)^T \quad (11.130)$$

The model becomes

$$\hat{\mathbf{y}} = \mathbf{W}^T \phi(x) \quad (11.131)$$

where $\mathbf{W}$ is a 2×2 matrix. Compute the MLE for $\mathbf{W}$ from the above data.

Exercise 11.2 [Centering and ridge regression]

Assume that $\bar{\mathbf{x}} = \mathbf{0}$, so the input data has been centered. Show that the optimizer of

$$J(\mathbf{w}, w_0) = (\mathbf{y} - \mathbf{X}\mathbf{w} - w_0\mathbf{1})^T (\mathbf{y} - \mathbf{X}\mathbf{w} - w_0\mathbf{1}) + \lambda \mathbf{w}^T \mathbf{w} \quad (11.132)$$

is

$$\hat{w}_0 = \bar{y} \quad (11.133)$$

$$\mathbf{w} = (\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^T \mathbf{y} \quad (11.134)$$

Exercise 11.3 [Partial derivative of the RSS †]

Let $RSS(\mathbf{w}) = \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2$ be the residual sum of squares.

a. Show that

$$\frac{\partial}{\partial w_k} RSS(\mathbf{w}) = a_k w_k - c_k \quad (11.135)$$

$$a_k = 2 \sum_{i=1}^n x_{ik}^2 = 2 \|\mathbf{x}_{:,k}\|^2 \quad (11.136)$$

$$c_k = 2 \sum_{i=1}^n x_{ik}(y_i - \mathbf{w}_{-k}^T \mathbf{x}_{i,-k}) = 2 \mathbf{x}_{:,k}^T \mathbf{r}_k \quad (11.137)$$

where $\mathbf{w}_{-k} = \mathbf{w}$ without component k , $\mathbf{x}_{i,-k}$ is $\mathbf{x}_i$ without component k , and $\mathbf{r}_k = \mathbf{y} - \mathbf{w}_{-k}^T \mathbf{x}_{:, -k}$ is the residual due to using all the features except feature k . Hint: Partition the weights into those involving k and those not involving k .

b. Show that if $\frac{\partial}{\partial w_k} RSS(\mathbf{w}) = 0$, then

$$\hat{w}_k = \frac{\mathbf{x}_{:,k}^T \mathbf{r}_k}{\|\mathbf{x}_{:,k}\|^2} \quad (11.138)$$

Hence when we sequentially add features, the optimal weight for feature k is computed by computing orthogonally projecting $\mathbf{x}_{:,k}$ onto the current residual.

Exercise 11.4 [Reducing elastic net to lasso]

Define

$$J_1(\mathbf{w}) = \|\mathbf{y} - \mathbf{X}\mathbf{w}\|^2 + \lambda_2 \|\mathbf{w}\|_2^2 + \lambda_1 \|\mathbf{w}\|_1 \quad (11.139)$$

and

$$J_2(\mathbf{w}) = \|\tilde{\mathbf{y}} - \tilde{\mathbf{X}}\mathbf{w}\|^2 + c\lambda_1 \|\mathbf{w}\|_1 \quad (11.140)$$

where $\|\mathbf{w}\|^2 = \|\mathbf{w}\|_2^2 = \sum_i w_i^2$ is the squared 2-norm, $\|\mathbf{w}\|_1 = \sum_i |w_i|$ is the 1-norm, $c = (1 + \lambda_2)^{-\frac{1}{2}}$, and

$$\tilde{\mathbf{X}} = c \begin{pmatrix} \mathbf{X} \\ \sqrt{\lambda_2} \mathbf{I}_d \end{pmatrix}, \quad \tilde{\mathbf{y}} = \begin{pmatrix} \mathbf{y} \\ \mathbf{0}_{d \times 1} \end{pmatrix} \quad (11.141)$$

Show

$$\operatorname{argmin} J_1(\mathbf{w}) = c(\operatorname{argmin} J_2(\mathbf{w})) \quad (11.142)$$

i.e.

$$J_1(c\mathbf{w}) = J_2(\mathbf{w}) \quad (11.143)$$

and hence that one can solve an elastic net problem using a lasso solver on modified data.

Exercise 11.5 [Shrinkage in linear regression †]

(Source: Jaakkola.) Consider performing linear regression with an orthonormal design matrix, so $\|\mathbf{x}_{:,k}\|_2^2 = 1$ for each column (feature) k , and $\mathbf{x}_{:,k}^T \mathbf{x}_{:,j} = 0$, so we can estimate each parameter w_k separately.

Figure 10.15b plots $\hat{w}_k$ vs $c_k = 2\mathbf{y}^T \mathbf{x}_{:,k}$, the correlation of feature k with the response, for 3 different estimation methods: ordinary least squares (OLS), ridge regression with parameter λ_2 , and lasso with parameter λ_1 .

- Unfortunately we forgot to label the plots. Which method does the solid (1), dotted (2) and dashed (3) line correspond to?
- What is the value of λ_1 ?
- What is the value of λ_2 ?

Exercise 11.6 [EM for mixture of linear regression experts]

Derive the EM equations for fitting a mixture of linear regression experts.